

Министерство науки и высшего образования Российской Федерации

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ «Национальный исследовательский университет ИТМО»
(Университет ИТМО)**

Факультет прикладной информатики

Образовательная программа Мобильные и сетевые технологии

Направление подготовки 09.03.03 Прикладная информатика

О Т Ч Е Т

по лабораторной работе №5

Тема задания: «Процедуры, функции, триггеры в PostgreSQL»

Обучающийся: Степанова Екатерина Андреевна, К3241

Руководитель: Говорова Марина Михайловна

Белов Александр Олегович

Санкт-Петербург,
2026

Цель работы: овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Оборудование: компьютерный класс.

Программное обеспечение: СУБД PostgreSQL, SQL Shell (psql).

Практическое задание (min - 6 баллов, max - 10 баллов, доп. балл3):

1. Создать 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР 2). Допустимо использование IN/INOUT параметров. Допустимо создать авторские процедуры. (3 балла)
2. Создать триггеры для индивидуальной БД согласно варианту:
Вариант 2.1. 10 триггеров по И3.
Вариант 2.2. 7 оригинальных триггеров + модификация триггерной функции Части 2 практического занятия доп. баллы).

Дополнительные баллы - 3:

Модифицировать триггер (триггерную функцию) на проверку корректности входа и выхода сотрудника (см. Практическое задание 1 Лабораторного практикума (Приложение)) с максимальным учетом «узких» мест некорректных данных по входу и выходу).

ВЫПОЛНЕНИЕ

Задание 1 Создать 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР 2). Допустимо использование IN/INOUT параметров. Допустимо создать авторские процедуры. (3 балла)

Для повышения стипендии отличникам на 10%.

Выведем необходимую первоначальную информацию

```
postgres=# select * from session.grant_type;
```

id_grant_type	grant_name	grant_size
4	Государственная академическая стипендия (ГАС)	2100
5	Повышенная государственная академическая стипендия (ПГАС)	30632
6	Стипендия от университета (ITMO FAMILY)	45632
7	Стипендия Президента РФ	30000
12	Стипендия Правительства РФ	20000
13	Именная стипендия Правительства Санкт-Петербурга	7000
14	Стипендия Фонда Владимира Потанина	25000
15	Стипендия Фонда им. В.И. Вернадского	5000
17	Социальная	13919

(9 строк)

```
postgres=# select * from session.grant;
```

grant_id	fk_id_grant_type	grant_year	fk_student_id
1	4	2023-09-01	550e8400-e29b-41d4-a716-446655440001
2	4	2024-09-01	550e8400-e29b-41d4-a716-446655440001
3	5	2024-09-01	550e8401-e29b-41d4-a716-446655440000
5	14	2024-09-01	550e8406-e29b-41d4-a716-446655440005
7	4	2024-09-01	550e8400-e29b-41d4-a716-446655440001
8	5	2024-09-01	550e8401-e29b-41d4-a716-446655440000
10	4	2024-09-01	550e8406-e29b-41d4-a716-446655440005

(7 строк)

Создадим процедуру по заданию

```
1. CREATE OR REPLACE PROCEDURE session.increase_scholarship()
2. LANGUAGE plpgsql
3. AS $$
4. DECLARE
5.     student RECORD;
6.     exam_total INTEGER;
7.     exam_excellent INTEGER;
8.     old_money NUMERIC;
9.     new_money NUMERIC;
10. BEGIN
11.     FOR student IN
12.         SELECT s.student_id, s.student_surname, s.student_name
13.         FROM session.student s
14.         WHERE s.student_id IN (SELECT fk_student_id FROM session."grant")
15.         AND s.status = 'Обучается'      -- только обучающиеся студенты
16.     LOOP
17.         SELECT COUNT(*), COUNT(CASE WHEN midterm_assessment >= 90 THEN 1 END)
18.         INTO exam_total, exam_excellent
19.         FROM session.passing_the_exam
20.         WHERE fk_student_id = student.student_id;
21.
22.         IF exam_total = exam_excellent AND exam_total > 0 THEN
23.             SELECT gt.grant_size INTO old_money
```

```

24.         FROM session."grant" g
25.         JOIN session.grant_type gt ON g.fk_id_grant_type = gt.id_grant_type
26.         WHERE g.fk_student_id = student.student_id
27.         LIMIT 1;
28.
29.         new_money = old_money * 1.10;
30.
31.         UPDATE session.grant_type
32.         SET grant_size = new_money
33.         WHERE id_grant_type IN (
34.             SELECT fk_id_grant_type
35.             FROM session."grant"
36.             WHERE fk_student_id = student.student_id
37.         );
38.
39.         RAISE NOTICE '%, %: стипендия повышена', student.student_surname,
student.student_name;
40.     END IF;
41. END LOOP;
42. END;
43. $$;

```

```

postgres=# call session.increase_scholarship();
ЗАМЕЧАНИЕ: Макаров, Дмитрий: стипендия повышена
ЗАМЕЧАНИЕ: Лебедев, Ольга: стипендия повышена
ЗАМЕЧАНИЕ: Королев, Мария: стипендия повышена
ЗАМЕЧАНИЕ: Васильев, Сергей: стипендия повышена
ЗАМЕЧАНИЕ: Семенов, Дарья: стипендия повышена
ЗАМЕЧАНИЕ: Васильев, Татьяна: стипендия повышена
ЗАМЕЧАНИЕ: Николаев, Юрий: стипендия повышена
ЗАМЕЧАНИЕ: Иванов, Ольга: стипендия повышена
ЗАМЕЧАНИЕ: Борисов, Валерия: стипендия повышена
ЗАМЕЧАНИЕ: Воробьев, Елена: стипендия повышена
ЗАМЕЧАНИЕ: Федоров, Алексей: стипендия повышена
ЗАМЕЧАНИЕ: Андреев, Илья: стипендия повышена
ЗАМЕЧАНИЕ: Яковлев, Дмитрий: стипендия повышена
ЗАМЕЧАНИЕ: Кузьмин, Анастасия: стипендия повышена
ЗАМЕЧАНИЕ: Яковлев, Дарья: стипендия повышена
ЗАМЕЧАНИЕ: Соловьев, София: стипендия повышена
ЗАМЕЧАНИЕ: Королев, Егор: стипендия повышена
ЗАМЕЧАНИЕ: Петров, Полина: стипендия повышена
ЗАМЕЧАНИЕ: Лебедев, Илья: стипендия повышена
CALL

```

```
CALL
postgres=# select * from session.grant_type;
 id_grant_type | grant_size | grant_name |
-----+-----+-----+
 4 | Государственная академическая стипендия (ГАС) | 2100 |
 6 | Стипендия от университета (ITMO FAMILY) | 45632 |
 7 | Стипендия Президента РФ | 30000 |
12 | Стипендия Правительства РФ | 20000 |
13 | Именная стипендия Правительства Санкт-Петербурга | 7000 |
14 | Стипендия Фонда Владимира Потанина | 25000 |
15 | Стипендия Фонда им. В.И. Вернадского | 5000 |
17 | Социальная | 13919 |
 5 | Повышенная государственная академическая стипендия (ПГАС) | 187342.5258615834381985912000000000000000000000 |
(9 строк)
```

Появилась проблема при таком запросе, что каждый раз при обнаружении отличника с определенной стипендией, эта стипендия многократно увеличивалась. Запрос неверный.

Новый запрос

Мы создадим отдельную ячейку со стипендией + 10%

Формируем название старое имя_10%

```
1. CREATE OR REPLACE PROCEDURE session.increase_scholarship()
2. LANGUAGE plpgsql
3. AS $$
4. DECLARE
5.     student RECORD;
6.     exam_total INTEGER;
7.     exam_excellent INTEGER;
8.     old_money NUMERIC;
9.     new_money NUMERIC;
10.    old_name VARCHAR;
11.    new_name VARCHAR;
12.    new_type_id INTEGER;
13. BEGIN
14.     FOR student IN
15.         SELECT s.student_id, s.student_surname, s.student_name
16.         FROM session.student s
17.         WHERE s.student_id IN (SELECT fk_student_id FROM session."grant")
18.         AND s.status = 'Обучается'
19.     LOOP
20.         SELECT COUNT(*), COUNT(CASE WHEN midterm_assessment >= 90 THEN 1 END)
21.         INTO exam_total, exam_excellent
```

```

22.         FROM session.passing_the_exam
23.         WHERE fk_student_id = student.student_id;
24.
25.         IF exam_total = exam_excellent AND exam_total > 0 THEN
26.             #Получаем текущую стипендию (размер и название)
27.             SELECT gt.grant_size, gt.grant_name INTO old_money, old_name
28.             FROM session."grant" g
29.             JOIN session.grant_type gt ON g.fk_id_grant_type = gt.id_grant_type
30.             WHERE g.fk_student_id = student.student_id
31.             LIMIT 1;
32.
33.             #Расчет новой стипендии
34.             new_money = old_money * 1.10;
35.
36.             #Назначаем новое название = старое название + " (+10%)"
37.             new_name = old_name || '(+10%)';
38.
39.             #Создаем новый тип стипендии (если еще не создан)
40.             INSERT INTO session.grant_type (grant_name, grant_size)
41.             VALUES (new_name, new_money)
42.             ON CONFLICT (grant_name)
43.             DO UPDATE SET grant_size = EXCLUDED.grant_size
44.             RETURNING id_grant_type INTO new_type_id;
45.
46.             #Переводим студента на новую стипендию
47.             UPDATE session."grant"
48.             SET fk_id_grant_type = new_type_id
49.             WHERE fk_student_id = student.student_id;
50.
51.             RAISE NOTICE '%, %: % -> % (%.0f -> %.0f)',
52.                 student.student_surname, student.student_name,
53.                 old_name, new_name, old_money, new_money;
54.         END IF;
55.     END LOOP;
56. END;
57. $$;

```

Перед применением процедуры, я исправлю значения текущих стипендий, так как ПГАС у нас невероятных размеров).

|| - конкатенация строк

Подробнее рассмотрим этот блок кода

```

58.
59.         INSERT INTO session.grant_type (grant_name, grant_size)
60.         VALUES (new_name, new_money)
61.         ON CONFLICT (grant_name)
62.         DO UPDATE SET grant_size = EXCLUDED.grant_size
63.         RETURNING id_grant_type INTO new_type_id;

```

Если кратко, то он пытается вставить новую запись, но если такая уже есть - обновляет существующую.

- 1) Пытаемся вставить новую запись с grant_name.
- 2) Проверяем, есть ли уже запись с таким именем
- 3) Если нет - вставляем новую запись, иначе - просто обновляем существующую

Сигнал отмены отправлен
 postgres=# select * from session.grant_type;

id_grant_type	grant_size	grant_name
4	2100	Государственная академическая стипендия (ГАС)
6	45632	Стипендия от университета (ITMO FAMILY)
7	30000	Стипендия Президента РФ
12	20000	Стипендия Правительства РФ
13	7000	Именная стипендия Правительства Санкт-Петербурга
14	25000	Стипендия Фонда Владимира Потанина
15	5000	Стипендия Фонда им. В.И. Вернадского
17	13919	Социальная
5	187342.5258615834381985912000000000000000000000	Повышенная государственная академическая стипендия (ПГАС)

(9 строк)

postgres=# update session.grant_type
 postgres=# set grant_size = 4000
 postgres=# where id_grant_type = 5;
 UPDATE 1
 postgres=# select * from session.grant_type;

id_grant_type	grant_size	grant_name
4	2100	Государственная академическая стипендия (ГАС)
6	45632	Стипендия от университета (ITMO FAMILY)
7	30000	Стипендия Президента РФ
12	20000	Стипендия Правительства РФ
13	7000	Именная стипендия Правительства Санкт-Петербурга
14	25000	Стипендия Фонда Владимира Потанина
15	5000	Стипендия Фонда им. В.И. Вернадского
17	13919	Социальная
5	4000	Повышенная государственная академическая стипендия (ПГАС)

(9 строк)

Обновила с помощью команды update, назначила ей размер 4000.

Применим новую процедуру.

```
postgres=# CALL session.increase_scholarship();
ЗАМЕЧАНИЕ: Макаров, Дмитрий: Повышенная государственная академическая стипендия (ПГАС) -> Повышенная государственная академическая стипендия (ПГАС) (+10%) (4000.0f -> 4400.00.0f)
ЗАМЕЧАНИЕ: Лебедев, Ольга: Повышенная государственная академическая стипендия (ПГАС) -> Повышенная государственная академическая стипендия (ПГАС) (+10%) (4000.0f -> 4400.00.0f)
ЗАМЕЧАНИЕ: Королев, Мария: Повышенная государственная академическая стипендия (ПГАС) -> Повышенная государственная академическая стипендия (ПГАС) (+10%) (4000.0f -> 4400.00.0f)
ЗАМЕЧАНИЕ: Васильев, Сергей: Повышенная государственная академическая стипендия (ПГАС) -> Повышенная государственная академическая стипендия (ПГАС) (+10%) (4000.0f -> 4400.00.0f)
ЗАМЕЧАНИЕ: Семенов, Дарья: Повышенная государственная академическая стипендия (ПГАС) -> Повышенная государственная академическая стипендия (ПГАС) (+10%) (4000.0f -> 4400.00.0f)
ЗАМЕЧАНИЕ: Васильев, Татьяна: Повышенная государственная академическая стипендия (ПГАС) -> Повышенная государственная академическая стипендия (ПГАС) (+10%) (4000.0f -> 4400.00.0f)
ЗАМЕЧАНИЕ: Николаев, Юрий: Повышенная государственная академическая стипендия (ПГАС) -> Повышенная государственная академическая стипендия (ПГАС) (+10%) (4000.0f -> 4400.00.0f)
ЗАМЕЧАНИЕ: Иванов, Ольга: Повышенная государственная академическая стипендия (ПГАС) -> Повышенная государственная академическая стипендия (ПГАС) (+10%) (4000.0f -> 4400.00.0f)
ЗАМЕЧАНИЕ: Борисов, Валерия: Повышенная государственная академическая стипендия (ПГАС) -> Повышенная государственная академическая стипендия (ПГАС) (+10%) (4000.0f -> 4400.00.0f)
ЗАМЕЧАНИЕ: Воробьев, Елена: Повышенная государственная академическая стипендия (ПГАС) -> Повышенная государственная академическая стипендия (ПГАС) (+10%) (4000.0f -> 4400.00.0f)
ЗАМЕЧАНИЕ: Федоров, Алексей: Повышенная государственная академическая стипендия (ПГАС) -> Повышенная государственная академическая стипендия (ПГАС) (+10%) (4000.0f -> 4400.00.0f)
ЗАМЕЧАНИЕ: Андреев, Илья: Повышенная государственная академическая стипендия (ПГАС) -> Повышенная государственная академическая стипендия (ПГАС) (+10%) (4000.0f -> 4400.00.0f)
ЗАМЕЧАНИЕ: Яковлев, Дмитрий: Повышенная государственная академическая стипендия (ПГАС) -> Повышенная государственная академическая стипендия (ПГАС) (+10%) (4000.0f -> 4400.00.0f)
ЗАМЕЧАНИЕ: Кузьмин, Анастасия: Повышенная государственная академическая стипендия (ПГАС) -> Повышенная государственная академическая стипендия (ПГАС) (+10%) (4000.0f -> 4400.00.0f)
ЗАМЕЧАНИЕ: Яковлев, Дарья: Повышенная государственная академическая стипендия (ПГАС) -> Повышенная государственная академическая стипендия (ПГАС) (+10%) (4000.0f -> 4400.00.0f)
ЗАМЕЧАНИЕ: Соловьев, София: Повышенная государственная академическая стипендия (ПГАС) -> Повышенная государственная академическая стипендия (ПГАС) (+10%) (4000.0f -> 4400.00.0f)
ЗАМЕЧАНИЕ: Королев, Егор: Повышенная государственная академическая стипендия (ПГАС) -> Повышенная государственная академическая стипендия (ПГАС) (+10%) (4000.0f -> 4400.00.0f)
ЗАМЕЧАНИЕ: Петров, Полина: Повышенная государственная академическая стипендия (ПГАС) -> Повышенная государственная академическая стипендия (ПГАС) (+10%) (4000.0f -> 4400.00.0f)
ЗАМЕЧАНИЕ: Лебедев, Илья: Повышенная государственная академическая стипендия (ПГАС) -> Повышенная государственная академическая стипендия (ПГАС) (+10%) (4000.0f -> 4400.00.0f)
(10 строк)
```

И самое главное - в таблице с видами стипендий не появилось лишних записей

```
postgres=# select * from session.grant_type;
 id_grant_type | grant_name | grant_size
-----+-----+-----
 4 | Государственная академическая стипендия (ГАС) | 2100
 6 | Стипендия от университета (ITMO FAMILY) | 45632
 7 | Стипендия Президента РФ | 30000
12 | Стипендия Правительства РФ | 20000
13 | Именная стипендия Правительства Санкт-Петербурга | 7000
14 | Стипендия Фонда Владимира Потанина | 25000
15 | Стипендия Фонда им. В.И. Вернадского | 5000
17 | Социальная | 13919
 5 | Повышенная государственная академическая стипендия (ПГАС) | 4000
18 | Повышенная государственная академическая стипендия (ПГАС) (+10%) | 4400.00
(10 строк)
```

Для перевода студентов на следующий курс.

Буду назначать новый курс, если они не выпускники, а также учитывать уровень образования и максимальный курс в нем. Также будут создаваться или назначаться новые группы с указанием курса в первой цифре группы.

```
1. CREATE OR REPLACE PROCEDURE session.transfer_students()
2. LANGUAGE plpgsql
3. AS $$
4. DECLARE
5.     stud RECORD;
6.     max_course INTEGER;
7.     new_course INTEGER;
```



```

8.     new_group_name VARCHAR;
9.     new_group_number UUID;
10. BEGIN
11.     FOR stud IN
12.         SELECT DISTINCT s.student_id, s.student_surname, s.student_name,
13.             g.course, g.group_number, g.group_name, g.fk_id_syllabus,
14.             s.fk_education_level
15.         FROM session.student s
16.         JOIN session."group" g ON s.fk_group_number = g.group_number
17.         WHERE s.status = 'Обучается'
18.     LOOP
19.         SELECT study_years INTO max_course
20.         FROM session.education_level
21.         WHERE id_level = stud.fk_education_level;
22.         new_course = stud.course + 1;
23.
24.         IF stud.course < max_course THEN
25.             #Формируем новое имя группы
26.             new_group_name = 'К' || new_course || SUBSTRING(stud.group_name, 3);
27.
28.             #Ищем группу
29.             SELECT group_number INTO new_group_number
30.             FROM session."group"
31.             WHERE group_name = new_group_name
32.             LIMIT 1;
33.
34.             #Если такой группы не существует, то формируем новую
35.             IF new_group_number IS NULL THEN
36.                 INSERT INTO session."group" (course, fk_id_syllabus, date_group_start,
37.                     date_group_finish, group_number, group_name)
38.                 VALUES (
39.                     new_course,
40.                     stud.fk_id_syllabus,
41.                     CURRENT_DATE,
42.                     CURRENT_DATE + INTERVAL '1 year',
43.                     gen_random_uuid(),
44.                     new_group_name
45.                 )
46.                 RETURNING group_number INTO new_group_number;
47.                 RAISE NOTICE 'Создана новая группа: %', new_group_name;
48.             END IF;
49.
50.             #Переводим студента
51.             UPDATE session.student
52.             SET fk_group_number = new_group_number
53.             WHERE student_id = stud.student_id;
54.             RAISE NOTICE '%, %: переведен в группу % (% курс)',
55.                 stud.student_surname, stud.student_name, new_group_name, new_course;
56.         ELSE
57.             RAISE NOTICE '%, %: ВЫПУСКНИК!', stud.student_surname, stud.student_name;
58.         END IF;
59.     END LOOP;
60.
61.     RAISE NOTICE 'Перевод завершен!';
62. END;
63. $$;

```

Выполнение процедуры

```
postgres=# call session.transfer_students();
ЗАМЕЧАНИЕ: Создана новая группа: K2090
ЗАМЕЧАНИЕ: Воробьев, Елена: переведен в группу K2090 (2 курс)
ЗАМЕЧАНИЕ: Александров, Мария: ВЫПУСКНИК!
ЗАМЕЧАНИЕ: Создана новая группа: K4242
ЗАМЕЧАНИЕ: Макаров, Сергей: переведен в группу K4242 (4 курс)
ЗАМЕЧАНИЕ: Создана новая группа: K2092
ЗАМЕЧАНИЕ: Соловьев, София: переведен в группу K2092 (2 курс)
ЗАМЕЧАНИЕ: Алексеев, Тимофей: ВЫПУСКНИК!
ЗАМЕЧАНИЕ: Создана новая группа: K4092
ЗАМЕЧАНИЕ: Романов, Виктория: переведен в группу K4092 (4 курс)
ЗАМЕЧАНИЕ: Создана новая группа: K2242
ЗАМЕЧАНИЕ: Васильев, Алексей: переведен в группу K2242 (2 курс)
ЗАМЕЧАНИЕ: Яковлев, Дмитрий: переведен в группу K3092 (3 курс)
ЗАМЕЧАНИЕ: Борисов, Валерия: ВЫПУСКНИК!
ЗАМЕЧАНИЕ: Григорьев, Дарья: переведен в группу K2092 (2 курс)
ЗАМЕЧАНИЕ: Степанов, Светлана: ВЫПУСКНИК!
ЗАМЕЧАНИЕ: Михайлов, Иван: переведен в группу K4242 (4 курс)
ЗАМЕЧАНИЕ: Королев, Мария: переведен в группу K2242 (2 курс)
ЗАМЕЧАНИЕ: Александров, Елизавета: переведен в группу K2242 (2 курс)
ЗАМЕЧАНИЕ: Макаров, Полина: ВЫПУСКНИК!
ЗАМЕЧАНИЕ: Павлов, Максим: ВЫПУСКНИК!
ЗАМЕЧАНИЕ: Сергеев, Вероника: переведен в группу K2242 (2 курс)
ЗАМЕЧАНИЕ: Попов, Алина: переведен в группу K3242 (3 курс)
ЗАМЕЧАНИЕ: Королев, Владимир: ВЫПУСКНИК!
ЗАМЕЧАНИЕ: Кузьмин, София: ВЫПУСКНИК!
ЗАМЕЧАНИЕ: Васильев, Даниил: ВЫПУСКНИК!
ЗАМЕЧАНИЕ: Фролов, Дарья: ВЫПУСКНИК!
ЗАМЕЧАНИЕ: Егоров, Михаил: ВЫПУСКНИК!
ЗАМЕЧАНИЕ: Гусев, Дмитрий: переведен в группу K2242 (2 курс)
ЗАМЕЧАНИЕ: Лебедев, Максим: переведен в группу K2092 (2 курс)
ЗАМЕЧАНИЕ: Алексеев, Алексей: переведен в группу K4092 (4 курс)
ЗАМЕЧАНИЕ: Григорьев, Тимофей: ВЫПУСКНИК!
ЗАМЕЧАНИЕ: Гусев, Валерия: переведен в группу K4092 (4 курс)
ЗАМЕЧАНИЕ: Федоров, Алексей: переведен в группу K3090 (3 курс)
ЗАМЕЧАНИЕ: Захаров, Татьяна: ВЫПУСКНИК!
ЗАМЕЧАНИЕ: Васильев, Сергей: ВЫПУСКНИК!
ЗАМЕЧАНИЕ: Семенов, Матвей: ВЫПУСКНИК!
ЗАМЕЧАНИЕ: Королев, Юлия: ВЫПУСКНИК!
ЗАМЕЧАНИЕ: Смирнов, Юлия: переведен в группу K4242 (4 курс)
```

Для изменения оценки при успешной пересдаче экзамена.

В данном запросе, я поняла, что структура моей таблицы меня только запутала в данном случае. Т.к. у меня оценка распределялась по 3м попыткам и ничего изменять как будто бы не нужно, так как при полученной положительной оценке это бы все равно учитывалось.

решила ввести новый столбец для максимальной положительной оценки(итоговая по предмету).

Назначу изначально всем 0.

В консоли не совсем видно, поэтому покажу результат в PgAdmin.

	id_passing_the_exam [PK] integer	fk_student_id uuid	midterm_assessment integer	re-interi_assessment_1 integer	re-interim asseessment_2 integer	m_a_date_block date	re_a_1 date	re_a_2 date	fk_teacher uuid	best_grade integer
1		550e8400-e29b-41d4-a716-4466554400...	80	[default]	[default]	2024-01-15	2024-01-...	2024-01-...	550e8400-e29b-41d4-a716-4466554400...	0
2		550e8400-e29b-41d4-a716-4466554400...	100	[default]	[default]	2024-01-20	2024-01-...	2024-01-...	550e8400-e29b-41d4-a716-4466554400...	0
3		550e8401-e29b-41d4-a716-4466554400...	59	[default]	[default]	2024-01-18	2024-01-...	2024-01-...	550e8400-e29b-41d4-a716-4466554400...	0
4		550e8404-e29b-41d4-a716-4466554400...	60	48	60	2024-01-20	2024-02-...	2024-02-...	550e8400-e29b-41d4-a716-4466554400...	0
5		550e8404-e29b-41d4-a716-4466554400...	64	[default]	[default]	2024-01-25	2024-01-...	2024-01-...	550e8400-e29b-41d4-a716-4466554400...	0
6		550e8406-e29b-41d4-a716-4466554400...	20	[default]	[default]	2024-01-16	2024-01-...	2024-01-...	550e8400-e29b-41d4-a716-4466554400...	0
7		550e8406-e29b-41d4-a716-4466554400...	90	[default]	[default]	2024-01-22	2024-01-...	2024-01-...	550e8400-e29b-41d4-a716-4466554400...	0
8		550e8408-e29b-41d4-a716-4466554400...	88	59	[default]	2024-01-19	2024-02-...	2024-02-...	550e8400-e29b-41d4-a716-4466554400...	0
9		550e8408-e29b-41d4-a716-4466554400...	50	[default]	[default]	2024-01-24	2024-01-...	2024-01-...	550e8400-e29b-41d4-a716-4466554400...	0
10	1222	550e8400-e29b-41d4-a716-4466554400...	35	35	0	2024-01-15	2024-01-...	2024-01-...	550e8400-e29b-41d4-a716-4466554400...	0
11	1223	41c8ee8f5-1141-4644-a978-dbf4c297e1...	95	0	0	2025-01-15	2025-01-...	2025-01-...	550e8400-e29b-41d4-a716-4466554400...	0
12	1224	aa40fc41-1ce2-45ed-af86-6500335ee504	92	0	0	2025-01-16	2025-01-...	2025-01-...	550e8400-e29b-41d4-a716-4466554400...	0
13	1225	3ae94476-19c4-4a56-962f-a5cdc92d93...	98	0	0	2025-01-17	2025-01-...	2025-01-...	550e8400-e29b-41d4-a716-4466554400...	0
14	1226	7664c2cf-7a5f-4ca1-899a-98969471f099	94	0	0	2025-01-18	2025-01-...	2025-01-...	550e8400-e29b-41d4-a716-4466554400...	0
15	1227	589e9a04-852-414e-b656-034422e8e...	96	0	0	2025-01-15	2025-01-...	2025-01-...	550e8400-e29b-41d4-a716-4466554400...	0
16	1228	2e1f937a-c7d0-4a23-a884-559f30a6900d	93	0	0	2025-01-16	2025-01-...	2025-01-...	550e8400-e29b-41d4-a716-4466554400...	0
17	1229	00d7605f-7dc5-43f6-bad5-095450b439f4	91	0	0	2025-01-17	2025-01-...	2025-01-...	550e8400-e29b-41d4-a716-4466554400...	0
18	1230	2c71bfac-480a-48dc-b0b5-5ecce84d523...	97	0	0	2025-01-15	2025-01-...	2025-01-...	550e8400-e29b-41d4-a716-4466554400...	0
19	1231	307a4832-bc40-439f-9627-a4887c1937...	95	0	0	2025-01-16	2025-01-...	2025-01-...	550e8400-e29b-41d4-a716-4466554400...	0
20	1232	958bdf2b-b890-4ae1-bbc8-7280fb78c8f...	99	0	0	2025-01-17	2025-01-...	2025-01-...	550e8400-e29b-41d4-a716-4466554400...	0
21	1233	ff78319f-5733-49c3-bcd5-ffb228ea8496	94	0	0	2025-01-15	2025-01-...	2025-01-...	550e8400-e29b-41d4-a716-4466554400...	0
22	1234	13b7ad9b-2fd2-425e-bc6b-fb456b5b7a09	92	0	0	2025-01-16	2025-01-...	2025-01-...	550e8400-e29b-41d4-a716-4466554400...	0
23	1235	d62541d5-9f0-4d95-89da-0a5c36a245...	96	0	0	2025-01-17	2025-01-...	2025-01-...	550e8400-e29b-41d4-a716-4466554400...	0
24	1236	0a0a4271-4964-477d-b356-8ed0263ab4...	93	0	0	2025-01-15	2025-01-...	2025-01-...	550e8400-e29b-41d4-a716-4466554400...	0
25	1237	e187b8f8-1130-41f0-b932-bd625061a3...	98	0	0	2025-01-16	2025-01-...	2025-01-...	550e8400-e29b-41d4-a716-4466554400...	0
26	1238	030f2019-1844-4177-952c-c7ce310b583f	95	0	0	2025-01-17	2025-01-...	2025-01-...	550e8400-e29b-41d4-a716-4466554400...	0
27	1239	d4e2d544-ae75-429e-b71f-4a531110e97f	97	0	0	2025-01-18	2025-01-...	2025-01-...	550e8400-e29b-41d4-a716-4466554400...	0
28	1240	a7781db-0617-4492-8c80-bb4c776f2e67	100	0	0	2025-01-19	2025-01-...	2025-01-...	550e8400-e29b-41d4-a716-4466554400...	0
29	1241	550e8400-e29b-41d4-a716-4466554400...	85	0	0	2025-01-10	2025-01-...	2025-01-...	550e8400-e29b-41d4-a716-4466554400...	0

Я решила сделать процедуру для изменения оценки конкретного студента, так как это зачастую требуется.

```
1. CREATE OR REPLACE PROCEDURE session.change_grade (  
2.     p_student_id UUID,  
3.     p_new_grade INTEGER,  
4.     p_retake_number INTEGER  
5. )  
6. LANGUAGE plpgsql  
7. AS $$  
8. BEGIN  
9.     IF p_new_grade < 0 OR p_new_grade > 100 THEN  
10.         RAISE EXCEPTION 'Оценка должна быть от 0 до 100';  
11.     END IF;  
12.     IF p_retake_number = 1 THEN  
13.         UPDATE session.passing_the_exam  
14.             SET "re-interi_asseessment_1" = p_new_grade  
15.             WHERE fk_student_id = p_student_id;  
16.  
17.     ELSIF p_retake_number = 2 THEN  
18.         UPDATE session.passing_the_exam  
19.             SET "re-interim asseessment_2" = p_new_grade  
20.             WHERE fk_student_id = p_student_id;  
21.  
22.     ELSE  
23.         RAISE EXCEPTION 'Номер пересдачи должен быть 1 или 2';  
24.     END IF;  
25.  
26.     UPDATE session.passing_the_exam  
27.     SET best_grade = GREATEST(  
28.         COALESCE(midterm_assessment, 0),  
29.         COALESCE("re-interi_asseessment_1", 0),  
30.         COALESCE("re-interim asseessment_2", 0)  
31.     )  
32.     WHERE fk_student_id = p_student_id;  
33.  
34.     RAISE NOTICE 'Студент %: новая оценка %, лучшая оценка стала %',  
35.         p_student_id, p_new_grade,
```

```

36.         (SELECT best_grade FROM session.passing_the_exam WHERE fk_student_id =
           p_student_id);
37. END;
38. $$;

```

Посмотрим всех студентов и их оценки(попытки и итоговая отметка)

```

postgres=# SELECT s.student_surname, s.student_name,
postgres=#         s.student_id,
postgres=#         pt.midterm_assessment,
postgres=#         pt."re-interim assessment_1",
postgres=#         pt."re-interim assessment_2",
postgres=#         pt.best_grade
postgres=# FROM session.student s
postgres=# JOIN session.passing_the_exam pt ON s.student_id = pt.fk_student_id
postgres=# WHERE s.status = 'Обучается'
postgres=# ORDER BY s.student_surname;
student_surname | student_name | student_id | midterm_assessment | re-interi-asseessment_1 | re-interim assessment_2 | best_gra
de
-----+-----+-----+-----+-----+-----+-----
Александров | Мария | 0a0a4271-4964-477d-b356-8ed0263ab442 | 84 | 0 | 0 |
Александров | Мария | 0a0a4271-4964-477d-b356-8ed0263ab442 | 93 | 0 | 0 |
Андреев | Илья | 307a4832-bc40-4391-9627-a4887c1937f5 | 95 | 0 | 0 |
Борисов | Валерия | d62541d5-5ff0-4d95-89da-0a5c36a24530 | 96 | 0 | 0 |
Васильев | Алексей | 550e8404-e29b-41d4-a716-446655440003 | 82 | 0 | 0 |
Васильев | Алексей | 550e8404-e29b-41d4-a716-446655440003 | 60 | 48 | 60 |
Васильев | Алексей | 550e8404-e29b-41d4-a716-446655440003 | 64 | | |
Васильев | Сергей | 3ea2de69-4ed2-4585-a679-8bd635c08256 | 97 | 0 | 0 |
Васильев | Татьяна | 589e9a04-c852-414e-b656-034422e8e6dc | 96 | 0 | 0 |
Васильев | Алексей | 550e8404-e29b-41d4-a716-446655440003 | 91 | 0 | 0 |
Воробьев | Елена | 3ae94476-19c4-4a56-962f-a5cdc92d938c | 98 | 0 | 0 |
Григорьев | Кристина | 41c8ee85-1141-4644-a978-db7dc297e1ac | 79 | 0 | 0 |
Иванов | Ольга | aa40fc41-1ce2-45ed-af86-6500335ee504 | 92 | 0 | 0 |
-- Далее --

```

```

0
Тестов | Студент | 99999999-9999-9999-9999-999999999999 | 50 | 0 | 0 |
0

```

```

postgres=# SELECT student_id, student_surname, student_name
postgres=# WHERE student_surname = 'Тестов'
postgres=# AND student_name = 'Студент';
student_id | student_surname | student_name
-----+-----+-----
99999999-9999-9999-9999-999999999999 | Тестов | Студент
(1 строка)

```

Вызовем процедуру

```

postgres=# CALL session.change_grade('99999999-9999-9999-9999-999999999999',
65, 1);
ЗАМЕЧАНИЕ: Студент 99999999-9999-9999-9999-999999999999: новая оценка 65, л
учшая оценка стала 65
CALL

```

Все работает корректно

```

Тестов | Студент | 99999999-9999-9999-9999-999999999999 | 50 | 65 | 0 |
65
Федоров | Алексей | 13b7ad0b-2fd2-4a5a-bc6b-fb456b5b7a00 | 92 | 0 | 0 |

```

Дополнительные процедуры

Необходимо назначить итоговой оценкой лучшую положительную за 3 попытки

```
1. CREATE OR REPLACE PROCEDURE session.calculate_best_grade()
2. LANGUAGE plpgsql
3. AS $$
4. DECLARE
5.     student RECORD;
6.     old_best INTEGER;
7.     new_best INTEGER;
8.     updated_count INTEGER;
9. BEGIN
10.    updated_count = 0;
11.
12.    FOR student IN
13.        SELECT fk_student_id,
14.               midterm_assessment,
15.               "re-interi_asseessment_1",
16.               "re-interim asseessment_2",
17.               best_grade
18.        FROM session.passing_the_exam
19.    LOOP
20.        old_best = student.best_grade;
21.        new_best = GREATEST(
22.            COALESCE(student.midterm_assessment, 0),
23.            COALESCE(student."re-interi_asseessment_1", 0),
24.            COALESCE(student."re-interim asseessment_2", 0)
25.        );
26.        IF old_best != new_best THEN
27.            UPDATE session.passing_the_exam
28.            SET best_grade = new_best
29.            WHERE fk_student_id = student.fk_student_id;
30.
31.            updated_count = updated_count + 1;
32.        END IF;
33.    END LOOP;
34.
35.    RAISE NOTICE 'Количество обновленных записей: %', updated_count;
36. END;
37. $$;
```

```
postgres=# call session.calculate_best_grade();
ЗАМЕЧАНИЕ: Количество обновленных записей: 58
CALL
```

Проверим как обновилась таблица

```

postgres=# SELECT s.student_surname, s.student_name, pt.best_grade
postgres=# FROM session.student s
postgres=# JOIN session.passing_the_exam pt ON s.student_id = pt.fk_student_id
postgres=# ORDER BY s.student_surname;
 student_surname | student_name | best_grade
-----+-----+-----
Александров    | Мария       | 84
Александров    | Мария       | 84
Андреев        | Илья        | 95
Борисов        | Валерия     | 96
Васильев       | Сергей      | 97
Васильев       | Татьяна     | 96
Васильев       | Алексей     | 82
Васильев       | Алексей     | 82
Васильев       | Алексей     | 82
Васильев       | Алексей     | 82
Воробьев       | Елена       | 98
Воробьева      | Татьяна     | 68
Воробьева      | Татьяна     | 68
Григорьев      | Кристина    | 79
Григорьев      | Кристина    | 79
Григорьев      | Александр   | 87
Иванов         | Иван        | 78
Иванов         | Иван        | 78
Иванов         | Иван        | 78
Иванов         | Ольга       | 92
Иванов         | Иван        | 78
Иванов         | Иван        | 78
Королев        | Мария       | 100
Королев        | Егор        | 91
Королев        | Мария       | 100
Кузнецов       | Дмитрий     | 73
Кузнецов       | Дмитрий     | 73
Кузнецов       | Матвей      | 71
Кузьмин        | Анастасия   | 94
Курагова       | Светлана    | 86
Лебедев        | Максим      | 90
Лебедев        | Ольга       | 98
Лебедев        | Максим      | 90
Лебедев        | Максим      | 90

```

У всех все обновилась, так как изначально у всех= был назначен 0.

Восстановление студента

```

1. CREATE OR REPLACE PROCEDURE session.restore_student(
2.   p_student_id UUID
3. )
4. LANGUAGE plpgsql
5. AS $$
6. BEGIN
7.   UPDATE session.student
8.   SET status = 'Обучается'
9.   WHERE student_id = p_student_id;

```

```

10.
11. RAISE NOTICE 'Студент % восстановлен', p_student_id;
12. END;
13. $$;

```

Выведем людей, которые сейчас находятся в академе

```

postgres=# SELECT * FROM session.student
postgres=# WHERE status = 'Академ';

```

status	student_surname	student_name	student_patronymic	document_number	student_id	fk_education_level	fk_g
Академ	Петрова	Алина	Леонидовна	АБ2345	550e8400-e29b-41d4-a716-446655440002	1	
Академ	Кузнецов	Дмитрий	Сергеевич	KY7890	550e8402-e29b-41d4-a716-446655440001	3	
Академ	Новикова	Ольга	Игоревна	HO9812	550e8405-e29b-41d4-a716-446655440004	2	
Академ	Иванов	Иван	Иванович	260523	550e8400-e29b-41d4-a716-446655440001	1	c6366216-e6a4-
Академ	Морозов	Павел	Андреевич	MO3456	550e8406-e29b-41d4-a716-446655440005	3	c6366216-e6a4-
Академ	Курагова	Светлана	Алексеевна	HB1234	ed76d66e-6747-4335-9068-c27e10817227	1	c6366216-e6a4-
Академ	Романов	Артем	Кириллович	AB4014	61320074-0450-47c3-b5c5-fbedd83c0b6d	3	eeafc65b-1dc9-
Академ	Григорьев	Александр	Никитич	AB9386	8ddc9702-7805-433a-bda3-67c06539dc0c	3	083f5385-1676-
Академ	Петров	Михаил	Матвеевна	AB7807	2168bed2-34e4-4c90-a6c7-9b3fe42019b7	1	6908d384-25a9-
Академ	Кузнецов	Олег	Андреевна	AB0328	8e6875f9-c87b-4d87-8428-d3a53e2b2325	1	1eb665bd-b2e6-
Академ	Смирнов	Анастасия	Сергеевна	AB0451	32f7b5dd-eca1-4d81-a893-edc7add2f233	1	083f5385-1676-
Академ	Кузнецов	Светлана	Сергеевич	AB6555	e0604177-916e-48e7-b0d2-77adc7dfec22	2	f05bae76-d004-
Академ	Михайлов	Елизавета	Олеговна	AB9620	95b6fb7f-2148-44ca-9c93-7a5ea3048a14	1	1eb665bd-b2e6-
Академ	Александров	Андрей	Тимофеевич	AB8172	04fe0ecb-9fbf-4fc2-b762-47a59d996148	2	0d368012-0dc2-
Академ	Полов	Максим	Максимович	AB4745	443233ab-0e20-4ffe-a4c2-093f735ad9eb	1	
Академ	Дмитриев	Максим	Дмитриевич	AB4698	a25de6e9-a940-4275-aa74-a8061131b9dd	1	32da32d3-4f98-
Академ	Семенов	Юрий	Олегович	AB2306	ad497aa6-39ce-48c8-8bb6-9a03721b4269	1	50356bd0-c351-
Академ	Романов	Наталья	Алексеевич	AB3044	fe27d9c2-1d87-4c63-a68b-b59c0d1905a4	2	
Академ	Гусев	Роман	Дмитриевна	AB4724	d43615a2-7c74-4f1d-869f-cd44a12c7469	1	
Академ	Лебедев	Наталья	Алексеевна	AB3324	2d4f3ca4-355d-480a-929d-2c365d908f52	3	083f5385-1676-
Академ	Полов	Ирина	Артемович	AB2423	d657467e-adf5-49cc-a9a2-f813f7f249e3	3	50356bd0-c351-

Давайте восстановим студента Петрова Алину Леонидовну с uuid = 550e8400-e29b-41d4-a716-446655440002

```

2 21
ЗАМЕЧАНИЕ: Студент 550e8400-e29b-41d4-a716-446655440002 восстановлен
CALL
postgres=#

```

```

postgres=# select * from session.student where student_id='550e8400-e29b-41d4-a716-446655440002';

```

status	student_surname	student_name	student_patronymic	document_number	student_id	fk_education_level	fk_group_number
Обучается	Петрова	Алина	Леонидовна	АБ2345	550e8400-e29b-41d4-a716-446655440002	1	

(1 строка)

Процедура выполнена успешно.

Перевод студента в академ. отпуск

```

1. CREATE OR REPLACE PROCEDURE session.set_academic_leave (
2.     p_student_id UUID,
3.     p_reason VARCHAR DEFAULT 'По семейным обстоятельствам'
4. )
5. LANGUAGE plpgsql

```

```

6. AS $$
7. BEGIN
8.     -- Проверяем, существует ли студент
9.     IF NOT EXISTS (SELECT 1 FROM session.student WHERE student_id = p_student_id) THEN
10.         RAISE EXCEPTION 'Студент с ID % не найден', p_student_id;
11.     END IF;
12.
13.     -- Переводим в академический отпуск
14.     UPDATE session.student
15.     SET status = 'Академ'
16.     WHERE student_id = p_student_id;
17.
18.     -- Отключаем стипендию
19.     DELETE FROM session."grant"
20.     WHERE fk_student_id = p_student_id;
21.
22.     RAISE NOTICE 'Студент % переведен в академический отпуск. Причина: %',
23.         p_student_id, p_reason;
24. END;
25. $$;

```

Переведем в академ студента.

```

postgres=# select * from session.student where student_id = '0a0a4271-4964-477d-b356-8ed0263ab442';
 status | student_surname | student_name | student_patronymic | document_number | student_id | fk_education_level | f
-----+-----+-----+-----+-----+-----+-----+-----
Обучается | Александров | Мария | Ивановна | AB7907 | 0a0a4271-4964-477d-b356-8ed0263ab442 | 3 | 0d368012-0dc2-4063-91b7-e160e9870bcb
(1 строка)

```

Вызовем процедуру

```

postgres=# call session.set_academic_leave('0a0a4271-4964-477d-b356-8ed0263ab442', 'Болезнь');
ЗАМЕЧАНИЕ: Студент 0a0a4271-4964-477d-b356-8ed0263ab442 переведен в академический отпуск. Причина: Болезнь
CALL
postgres=#

```

```

postgres=# select * from session.student where student_id = '0a0a4271-4964-477d-b356-8ed0263ab442';
 status | student_surname | student_name | student_patronymic | document_number | student_id | fk_education_level | fk_g
-----+-----+-----+-----+-----+-----+-----+-----
Академ | Александров | Мария | Ивановна | AB7907 | 0a0a4271-4964-477d-b356-8ed0263ab442 | 3 | 0d368012-0dc2-4063-91b7-e160e9870bcb
(1 строка)

```


Задание 2

Создать триггеры для индивидуальной БД согласно варианту:

Вариант 2.1. 10 триггеров по ИЗ.

Вариант 2.2. 7 оригинальных триггеров + модификация триггерной функции Части 2 практического занятия доп. баллы).

Дополнительные баллы - 3:

Модифицировать триггер (триггерную функцию) на проверку корректности входа и выхода сотрудника (см. Практическое задание 1 Лабораторного практикума (Приложение)) с максимальным учетом «узких» мест некорректных данных по входу и выходу).

Задание 2.1 (Триггеры по ИЗ)

1) Проверка того, что время занятия корректное

Для Insert и Update, поэтому используем NEW

Имя	Тип	Описание
NEW	RECORD	Новые значения полей записи базы данных, созданной командой INSERT или обновленной командой UPDATE, при срабатывании триггера уровня записи (ROW). Переменная используется для модификации новых записей. Внимание !!! Переменная NEW доступна только при операциях INSERT и UPDATE. Поля записи NEW могут быть изменены триггером.

Синтаксис IF

MySQL 8.1

```
IF условие THEN
    -- код, выполняемый при истинном условии
ELSEIF другое_условие THEN
    -- код для альтернативного условия
ELSE
    -- код по умолчанию
END IF;
```

Создадим триггер

Создание триггера определяется командой:

```
CREATE TRIGGER триггер
BEFORE | AFTER } { событие [ OR событие ] } ON таблица
FOR EACH { ROW | STATEMENT }
EXECUTE PROCEDURE функция ( аргументы )
```

```
1. CREATE TRIGGER trg_check_schedule_time
2.     BEFORE INSERT OR UPDATE ON session.schedule
3.     FOR EACH ROW
4.     EXECUTE FUNCTION session.check_schedule_time();
```

Но перед этим необходимо создать саму функцию session.check_schedule_time().

```
1. CREATE FUNCTION session.check_schedule_time()
2. RETURNS TRIGGER AS $$
3. BEGIN
4.     IF NEW.time_start >= NEW.time_end THEN
5.         RAISE EXCEPTION 'Время начала должно быть раньше времени окончания!';
6.     END IF;
7.     RETURN NEW;
8. END;
9. $$ LANGUAGE plpgsql;
```

```
postgres=# CREATE TRIGGER trg_check_schedule_time
postgres=#     BEFORE INSERT OR UPDATE ON session.schedule
postgres=#     FOR EACH ROW
postgres=#     EXECUTE FUNCTION session.check_schedule_time();
ОШИБКА: функция session.check_schedule_time() не существует
postgres=# CREATE FUNCTION session.check_schedule_time()
postgres=# RETURNS TRIGGER AS $$
postgres$$ BEGIN
postgres$$     IF NEW.time_start >= NEW.time_end THEN
postgres$$         RAISE EXCEPTION 'Время начала должно быть раньше времени
окончания!';
postgres$$     END IF;
postgres$$     RETURN NEW;
postgres$$ END;
postgres$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION
postgres=#
postgres=# CREATE TRIGGER trg_check_schedule_time
postgres=#     BEFORE INSERT OR UPDATE ON session.schedule
postgres=#     FOR EACH ROW
postgres=#     EXECUTE FUNCTION session.check_schedule_time();
CREATE TRIGGER
```

Теперь проверим триггер

Выведем все возможные дисциплины и введем новое расписание для одной из них

```
postgres=# select * from session.educational_program;
```

id_educational_program	educational_program_name	date_edu_program	fk_field_of_stud
1	Разработка программного обеспечения и веб-технологий	2023-09-01	09.03.04
2	Мобильная и встраиваемая разработка	2023-09-01	09.03.04
3	Вычислительные системы и сети	2023-09-01	09.03.01
4	Системное администрирование и DevOps	2023-09-01	09.03.01
5	Защита информационных систем	2023-09-01	10.05.01
6	Криптографические методы защиты	2023-09-01	10.05.01
7	Архитектура корпоративных систем	2023-09-01	09.04.04
8	Управление IT-проектами	2023-09-01	09.04.04
9	Физика конденсированного состояния	2023-09-01	03.03.02
10	Квантовая оптика и лазерная физика	2023-09-01	03.03.02
11	Физика ядерных реакторов	2023-09-01	03.05.02
12	Наноэлектроника и нанотехнологии	2023-09-01	03.05.02
13	Алгоритмы и структуры данных	2023-09-01	02.03.02
14	Искусственный интеллект и машинное обучение	2023-09-01	02.03.02
15	Молекулярная биология и генетика	2023-09-01	06.03.01
16	Биоинформатика и системная биология	2023-09-01	06.03.01
17	Управление инновационными проектами	2023-09-01	38.03.02
18	Стратегический менеджмент	2023-09-01	38.03.02
19	Цифровые коммуникации и брендинг	2023-09-01	42.03.01
20	Медиапланирование и PR-аналитика	2023-09-01	42.03.01
21	Цифровое издательское дело	2023-09-01	42.03.03
22	Редакторская подготовка медиапроектов	2023-09-01	42.03.03
23	Защита информации в телекоммуникациях	2023-09-01	10.03.01
24	Информационная безопасность банковских систем	2023-09-01	10.03.01
25	Когнитивная психология	2023-09-01	37.03.01
26	Нейропсихология и психофизиология	2023-09-01	37.03.01
27	Цифровое право и правовая информатика	2023-09-01	40.03.01
28	Юридическое сопровождение IT-бизнеса	2023-09-01	40.03.01
29	Промышленные роботы и автоматизация	2023-09-01	15.03.06
30	Интеллектуальные мехатронные системы	2023-09-01	15.03.06

(30 строк)

Возьмем дисциплину с id = 1

```
postgres=# INSERT INTO session.schedule (
postgres(#      id_schedule, time_start, time_end, date,
postgres(#      fk_id_discipline_in_the_syllabus, fk_id_audience,
postgres(#      fk_id_teacher, fk_group_number
postgres(# ) VALUES (
postgres(#      DEFAULT, '10:00:00+03', '11:30:00+03', '2025-05-15',
postgres(#      1, 30, '550e8400-e29b-41d4-a716-446655440000',
postgres(#      'c6366216-e6a4-45ba-aeaf-9aa90e4e73ca'
postgres(# );
INSERT 0 1
```

Все корректно добавилось, так как у нас нет нарушения условия триггера.

А теперь попробуем дать некорректные данные.

```
postgres=# INSERT INTO session.schedule (
postgres(#      id_schedule, time_start, time_end, date,
postgres(#      fk_id_discipline_in_the_syllabus, fk_id_audience,
postgres(#      fk_id_teacher, fk_group_number
postgres(# ) VALUES (
postgres(#      DEFAULT, '12:00:00+03', '11:30:00+03', '2025-05-15',
postgres(#      1, 30, '550e8400-e29b-41d4-a716-446655440000',
postgres(#      'c6366216-e6a4-45ba-aeaf-9aa90e4e73ca'
postgres(# );
ОШИБКА:  Время начала должно быть раньше времени окончания!
КОНТЕКСТ:  функция PL/pgSQL session.check_schedule_time(), строка 4, оператор RAISE
```

Работает верно.

2) Проверка того, что группа существует при добавлении студентов

```
1. CREATE OR REPLACE FUNCTION session.check_group_exists()
2. RETURNS TRIGGER AS $$
3. BEGIN
4.     IF NOT EXISTS (SELECT 1 FROM session."group" WHERE group_number =
5.         NEW.fk_group_number) THEN
6.         RAISE EXCEPTION 'Группа с ID % не существует', NEW.fk_group_number;
7.     END IF;
8.     RETURN NEW;
9. $$ LANGUAGE plpgsql;
10.
11. CREATE TRIGGER trg_check_group_exists
12.     BEFORE INSERT OR UPDATE ON session.student
13.     FOR EACH ROW
14.     EXECUTE FUNCTION session.check_group_exists();
```

Посмотрим существующие группы

```
postgres=# select group_number, group_name from session.group;
-----+-----
c6366216-e6a4-45ba-aeaf-9aa90e4e73ca | K3090
eeafc65b-1dc0-49d3-b795-44e35a7ff668 | K3090
1eb665bd-b2e6-45b1-8309-a21a6b69bb88 | K3090
bb8a8b1b-0108-478a-859a-165fc640b180 | K3090
32da32d3-4f98-4db5-9bc5-91d0591deb95 | K3092
0d368012-0dc2-4063-91b7-e160e9870bcb | K3092
9c1f5da6-8e43-45c0-85bc-4cbc2832cb05 | K3092
a875bca0-d8a0-4a50-9c7a-6411c0e69d12 | K3092
6908d384-25a9-4421-a1c9-9deb1eccdaba | K3242
f05bae76-d004-4d00-83c6-5922789e3ffd | K3242
083f5385-1676-499c-ba88-208911a8c80a | K3242
50356bd0-c351-4a18-9e76-ac3359f96df8 | K3242
aa60ea56-415f-4b70-977e-db3ad3362e30 | K2090
c0c9cab7-49ea-4493-9bcf-655d8743ca71 | K4242
f0060f06-46e5-45a4-93c5-81123ede2707 | K2092
9684db21-09b0-4c74-ba7e-15ded794156c | K4092
f9f013f8-e044-4164-95c2-0f705431d9eb | K2242
3503b79c-84e0-4d84-8840-7cca591d82da | K4090
(18 строк)
```

Возьмем для корректного варианта группу K3090, а для некорректного запроса W1245.

```
postgres=# INSERT INTO session.student(student_id, status, student_surname,
student_name,
postgres(#      student_patronymic, document_number, fk_education_level,
postgres(#      fk_group_number) VALUES
postgres=#      (gen_random_uuid(), 'Обучается', 'Толстой', 'Лев',
postgres=#      'Николаевич', 'DOC21', 1,
postgres=#      'c6366216-e6a4-45ba-aeaf-9aa90e4e73ca');
INSERT 0 1
```

```
postgres=# select * from session.student where student_surname = 'Толстой';
 status | student_surname | student_name | student_patronymic | document_number | student_id | fk_education_level | f
-----+-----+-----+-----+-----+-----+-----+-----
Обучается | Толстой | Лев | Николаевич | DOC21 | 91b5d4ac-f8ab-475a-9cb8-262b2d060b60 | 1 | c6366216-e6
a4-45ba-aeaf-9aa90e4e73ca
(1 строка)
```

При ошибочных данных срабатывает корректно

```
postgres=# INSERT INTO session.student(student_id, status, student_surname,
student_name,
postgres(#      student_patronymic, document_number, fk_education_level,
postgres(#      fk_group_number) VALUES
postgres=#      (gen_random_uuid(), 'Обучается', 'Толстой', 'Лев',
postgres=#      'Николаевич', 'DOC21', 1,
postgres=#      'c6366216-e6a4-45ba-aeaf-9aa90e4e63ca');
ОШИБКА:  Группа с ID c6366216-e6a4-45ba-aeaf-9aa90e4e63ca не существует
КОНТЕКСТ:  функция PL/pgSQL session.check_group_exists(), строка 4, оператор
RAISE
```

3) Проверка того, что оценка положительное число и не больше 103 при добавлении

```
1. CREATE OR REPLACE FUNCTION session.check_grade()
2. RETURNS TRIGGER AS $$
3. BEGIN
4.     IF NEW.midterm_assessment < 0 OR NEW.midterm_assessment > 103 THEN
5.         RAISE EXCEPTION 'Оценка студента должна быть от 0 до 103 баллов!';
6.     END IF;
7.     RETURN NEW;
8. END;
9. $$ LANGUAGE plpgsql;
10.
11. CREATE TRIGGER trg_check_grade_range
12. BEFORE INSERT OR UPDATE ON session.passing_the_exam
13. FOR EACH ROW
14. EXECUTE FUNCTION session.check_grade();
```

```
postgres=# INSERT INTO session.passing_the_exam (
postgres(#      fk_student_id, midterm_assessment, m_a_date_block, re_a_1, re
_a_2, fk_teacher
postgres(# ) VALUES (
postgres(#      '550e8400-e29b-41d4-a716-446655440001', 85, '2025-05-20', '20
25-05-20', '2025-05-20',
postgres(#      '550e8400-e29b-41d4-a716-446655440000'
postgres(# );
INSERT 0 1
```

```
postgres=# INSERT INTO session.passing_the_exam (
postgres(#      fk_student_id, midterm_assessment, m_a_date_block, re_a_1, re
_a_2, fk_teacher
postgres(# ) VALUES (
postgres(#      '550e8400-e29b-41d4-a716-446655440001', 1000, '2025-05-20', '
2025-05-20', '2025-05-20',
postgres(#      '550e8400-e29b-41d4-a716-446655440000'
postgres(# );
ОШИБКА: Оценка студента должна быть от 0 до 103 баллов!
КОНТЕКСТ: функция PL/pgSQL session.check_grade(), строка 4, оператор RAISE
```

4) Запретим удалять группу, которая уже существует и в ней есть студенты

```
1. CREATE OR REPLACE FUNCTION session.no_delete_group_with_students()
2. RETURNS TRIGGER AS $$
3. DECLARE
4.     k INTEGER;
5. BEGIN
6.     SELECT COUNT(*) INTO k FROM session.student WHERE fk_group_number =
       OLD.group_number;
7.     IF k > 0 THEN
8.         RAISE EXCEPTION 'Нельзя удалить группу, в которой есть студенты (%)', k;
9.     END IF;
10.    RETURN OLD;
11. END;
12. $$ LANGUAGE plpgsql;
13.
14. CREATE TRIGGER trg_no_delete_group
15.     BEFORE DELETE ON session.group
16.     FOR EACH ROW
17.     EXECUTE FUNCTION session.no_delete_group_with_students();
```

```
postgres=# DELETE FROM session.group WHERE group_name = 'K3090';
ОШИБКА: Нельзя удалить группу, в которой есть студенты (12)
КОНТЕКСТ: функция PL/pgSQL session.no_delete_group_with_students(), строка
7, оператор RAISE
```

А теперь добавим какую-нибудь группу, в которой нет студентов.

```
postgres=# INSERT INTO session.group (course, fk_id_syllabus, date_group_start, date_group_finish, group_number, group_name)
postgres=# VALUES (1, 1, '2026-05-01', '2026-12-31', gen_random_uuid(), 'Q7090');
INSERT 0 1
```

```
postgres=# DELETE FROM session.group WHERE group_name = 'Q7090';
DELETE 1
postgres=#
```

5) Можно ввести автоматическое обновление best-grade

С помощью этого триггера можно улучшить процедуру “Необходимо назначить итоговой оценкой лучшую положительную за 3 попытки”

```
1. CREATE OR REPLACE FUNCTION session.update_best_grade()
2. RETURNS TRIGGER AS $$
3. BEGIN
4.     NEW.best_grade = GREATEST(
5.         COALESCE(NEW.midterm_assessment, 0),
6.         COALESCE(NEW."re-interi_asseessment_1", 0),
7.         COALESCE(NEW."re-interim asseessment_2", 0)
8.     );
9.     RETURN NEW;
10. END;
11. $$ LANGUAGE plpgsql;
12.
13. CREATE TRIGGER trg_update_best_grade
14. BEFORE INSERT OR UPDATE ON session.passing_the_exam
15. FOR EACH ROW
16. EXECUTE FUNCTION session.update_best_grade();
```

```
postgres=# INSERT INTO session.passing_the_exam (
postgres=#     fk_student_id, midterm_assessment, "re-interi_asseessment_1",
postgres=#     "re-interim asseessment_2", m_a_date_block, re_a_1, re_a_2, f
k_teacher
postgres=# ) VALUES (
postgres=#     '550e8400-e29b-41d4-a716-446655440001', 75, 85, 95,
postgres=#     '2025-05-20', '2025-05-20', '2025-05-20',
postgres=#     '550e8400-e29b-41d4-a716-446655440000'
postgres=# ) RETURNING id_passing_the_exam, midterm_assessment, "re-interi_a
sseessment_1",
postgres=#     "re-interim asseessment_2", best_grade;
 id_passing_the_exam | midterm_assessment | re-interi_asseessment_1 | re-int
erim asseessment_2 | best_grade
-----+-----+-----+-----+-----
-----+-----+-----+-----+-----
1277 | 75 | 85 | 95 |
95 | 95 | 75 | 85 |
(1 строка)
```

6) Запрет вставки занятия в прошлом

```
1. CREATE OR REPLACE FUNCTION session.check_date_not_past()
2. RETURNS TRIGGER AS $$
3. BEGIN
4.     IF NEW.date < CURRENT_DATE THEN
5.         RAISE EXCEPTION 'Это событие (дата: %) уже прошло', NEW.date;
6.     END IF;
7.     RETURN NEW;
8. END;
9. $$ LANGUAGE plpgsql;
```

```

10.
11. CREATE TRIGGER trg_check_date_not_past
12.     BEFORE INSERT ON session.schedule
13.     FOR EACH ROW
14.     EXECUTE FUNCTION session.check_date_not_past();

```

Занятие на будущую дату

```

postgres=# INSERT INTO session.schedule (
postgres(#      time_start, time_end, date,
postgres(#      fk_id_discipline_in_the_syllabus, fk_id_audience,
postgres(#      fk_id_teacher, fk_group_number
postgres(# ) VALUES (
postgres(#      '10:00:00+03', '11:30:00+03', CURRENT_DATE + INTERVAL '30 day
s',
postgres(#      1, 30, '550e8400-e29b-41d4-a716-446655440000',
postgres(#      'c6366216-e6a4-45ba-aeaf-9aa90e4e73ca'
postgres(# ) RETURNING id_schedule, date;
id_schedule |      date
-----+-----
          51 | 2026-05-31
(1 строка)

```

Занятие на прошедшую дату

```

postgres=# INSERT INTO session.schedule (
postgres(#      time_start, time_end, date,
postgres(#      fk_id_discipline_in_the_syllabus, fk_id_audience,
postgres(#      fk_id_teacher, fk_group_number
postgres(# ) VALUES (
postgres(#      '10:00:00+03', '11:30:00+03', CURRENT_DATE - INTERVAL '30 day
s',
postgres(#      1, 30, '550e8400-e29b-41d4-a716-446655440000',
postgres(#      'c6366216-e6a4-45ba-aeaf-9aa90e4e73ca'
postgres(# ) RETURNING id_schedule, date;
ОШИБКА: Это событие (дата: 2026-04-01) уже прошло
КОНТЕКСТ: функция PL/pgSQL session.check_date_not_past(), строка 4, оператор RAISE

```

7) Запрет наложения расписания

```

1. CREATE OR REPLACE FUNCTION session.no_duplicate_schedule()
2. RETURNS TRIGGER AS $$
3. BEGIN
4.     IF EXISTS (
5.         SELECT 1 FROM session.schedule
6.         WHERE fk_id_audience = NEW.fk_id_audience
7.             AND date = NEW.date
8.             AND time_start = NEW.time_start
9.             AND (TG_OP = 'INSERT' OR id_schedule != NEW.id_schedule)
10.    ) THEN
11.        RAISE EXCEPTION 'В этой аудитории уже есть занятие в указанное время';
12.    END IF;
13.    RETURN NEW;

```



```

14. END;
15. $$ LANGUAGE plpgsql;
16.
17. CREATE TRIGGER trg_no_duplicate_schedule
18.     BEFORE INSERT OR UPDATE ON session.schedule
19.     FOR EACH ROW
20.     EXECUTE FUNCTION session.no_duplicate_schedule();

```

```

postgres=# UPDATE session.schedule
postgres=# SET time_start = '09:00:00'
postgres=# WHERE id_schedule = (SELECT id_schedule FROM session.schedule LIM
IT 1);
UPDATE 1

```

```

postgres=# INSERT INTO session.schedule (time_start, time_end, date, fk_id_d
iscipline_in_the_syllabus,
postgres=#     fk_id_audience, fk_id_teacher, fk_group_number)
postgres=# VALUES (
postgres=#     '09:00:00', -- время начала
postgres=#     '10:30:00', -- время окончания
postgres=#     '2024-09-02', -- дата
postgres=#     1, -- discipline
postgres=#     (SELECT fk_id_audience FROM session.schedule LIMIT 1), -- та
же аудитория
postgres=#     (SELECT id_teacher FROM session.teacher LIMIT 1),
postgres=#     (SELECT group_number FROM session."group" LIMIT 1)
postgres=# );
ОШИБКА: Это событие (дата: 2024-09-02) уже прошло
КОНТЕКСТ: функция PL/pgSQL session.check_date_not_past(), строка 4, операто
р RAISE

```

8) Проверка того, что преподаватель существует

```

1. CREATE OR REPLACE FUNCTION session.check_teacher_exists()
2. RETURNS TRIGGER AS $$
3. BEGIN
4.     IF NOT EXISTS (SELECT 1 FROM session.teacher WHERE id_teacher = NEW.fk_id_teacher)
5.     THEN
6.         RAISE EXCEPTION 'Преподаватель с ID % не найден', NEW.fk_id_teacher;
7.     END IF;
8.     RETURN NEW;
9. $$ LANGUAGE plpgsql;
10.
11. CREATE TRIGGER trg_check_teacher_exists
12.     BEFORE INSERT OR UPDATE ON session.schedule
13.     FOR EACH ROW
14.     EXECUTE FUNCTION session.check_teacher_exists();

```

Сделаем вставку с существующим преподавателем.

Для начала выведем uuid некоторых существующих преподавателей.

Возьмем для примера первый в строке uuid.

```
postgres=# select id_teacher from session.teacher limit 2;
id_teacher
-----
550e8400-e29b-41d4-a716-446655440000
550e8400-e29b-41d4-a716-446655440001
(2 строки)
```

Добавим расписание с ним, как мы видим из запроса ниже - все прошло успешно.

```
postgres=# INSERT INTO session.schedule (
postgres(#      time_start, time_end, date,
postgres(#      fk_id_discipline_in_the_syllabus, fk_id_audience,
postgres(#      fk_id_teacher, fk_group_number
postgres(# ) VALUES (
postgres(#      '10:00:00+03', '11:30:00+03', CURRENT_DATE + INTERVAL '10 day
s',
postgres(#      1, 30, '550e8400-e29b-41d4-a716-446655440000',
postgres(#      'c6366216-e6a4-45ba-aeaf-9aa90e4e73ca'
postgres(# );
INSERT 0 1
```

А теперь придумаем uuid для несуществующего преподавателя.

```
postgres=# INSERT INTO session.schedule (
postgres(#      time_start, time_end, date,
postgres(#      fk_id_discipline_in_the_syllabus, fk_id_audience,
postgres(#      fk_id_teacher, fk_group_number
postgres(# ) VALUES (
postgres(#      '10:00:00+03', '11:30:00+03', CURRENT_DATE + INTERVAL '10 day
s',
postgres(#      1, 30, '00000000-0000-0000-0000-000000000000', -- несуществу
ющий UUID
postgres(#      'c6366216-e6a4-45ba-aeaf-9aa90e4e73ca'
postgres(# );
ОШИБКА: Преподаватель с ID 00000000-0000-0000-0000-000000000000 не найден
КОНТЕКСТ: функция PL/pgSQL session.check_teacher_exists(), строка 4, операт
ор RAISE
```

Произошла ошибка.

9) Проверка того, что аудитория существует

```
1. CREATE OR REPLACE FUNCTION session.check_audience_exists()
2. RETURNS TRIGGER AS $$
3. BEGIN
4.     IF NOT EXISTS (SELECT 1 FROM session.audience WHERE id_audience =
NEW.fk_id_audience) THEN
5.         RAISE EXCEPTION 'Аудитория с ID % не найдена', NEW.fk_id_audience;
6.     END IF;
7.     RETURN NEW;
8. END;
9. $$ LANGUAGE plpgsql;
10.
11. CREATE TRIGGER trg_check_audience_exists
12.     BEFORE INSERT OR UPDATE ON session.schedule
```

```

13.     FOR EACH ROW
14.     EXECUTE FUNCTION session.check_audience_exists();

```

Insert с существующей аудиторией

Попыталась вставить, но забыла, что в это время есть уже занятие в этой аудитории. Сработал соответствующий триггер

```

CREATE TRIGGER
postgres=# INSERT INTO session.schedule (
postgres(#      time_start, time_end, date,
postgres(#      fk_id_discipline_in_the_syllabus, fk_id_audience,
postgres(#      fk_id_teacher, fk_group_number
postgres(# ) VALUES (
postgres(#      '10:00:00+03', '11:30:00+03', CURRENT_DATE + INTERVAL '10 day
s',
postgres(#      1, 30, '550e8400-e29b-41d4-a716-446655440000',
postgres(#      'c6366216-e6a4-45ba-aeaf-9aa90e4e73ca'
postgres(# );
ОШИБКА: В этой аудитории уже есть занятие в указанное время
КОНТЕКСТ: функция PL/pgSQL session.no_duplicate_schedule(), строка 10, опер
атор RAISE

```

Попробуем еще раз

```

postgres=# SELECT * FROM session.audience;
 id_audience |      audienc_number      | capacity | fk_area | audience_floo
r
-----+-----+-----+-----+-----
--
      25 | Столовая                  |    300 |      2 |
1      26 | Туалет                    |     10 |      2 |
1      27 | Приемная комиссия        |     20 |      2 |
1      28 | Гардероб                  |     40 |      2 |
1      29 | Конференц-зал             |     20 |      2 |
3      30 | 1305                      |     60 |      2 |
3      31 | Lemon Classroom           |    100 |      2 |
4      32 | 1412                      |     20 |      2 |
4      33 | 1524                      |     40 |      2 |
5      34 | 1519                      |     40 |      2 |
5

```

Поставим занятие в аудитории 1524

```

postgres=# INSERT INTO session.schedule (
postgres#     time_start, time_end, date,
postgres#     fk_id_discipline_in_the_syllabus, fk_id_audience,
postgres#     fk_id_teacher, fk_group_number
postgres# ) VALUES (
postgres#     '8:00:00+03', '9:50:00+03', CURRENT_DATE + INTERVAL '10 days'
postgres#     2, 33, '550e8400-e29b-41d4-a716-446655440000',
postgres#     'c6366216-e6a4-45ba-aeaf-9aa90e4e73ca'
postgres# );
INSERT 0 1

```

А теперь добавим занятие в несуществующую аудиторию

```

postgres=# INSERT INTO session.schedule (
postgres#     time_start, time_end, date,
postgres#     fk_id_discipline_in_the_syllabus, fk_id_audience,
postgres#     fk_id_teacher, fk_group_number
postgres# ) VALUES (
postgres#     '8:00:00+03', '9:50:00+03', CURRENT_DATE + INTERVAL '10 days'
postgres#     4, 705, '550e8400-e29b-41d4-a716-446655440000',
postgres#     'c6366216-e6a4-45ba-aeaf-9aa90e4e73ca'
postgres# );
ОШИБКА: Аудитория с ID 705 не найдена
КОНТЕКСТ: функция PL/pgSQL session.check_audience_exists(), строка 4, опера
тор RAISE

```

10) Проверка того, что аудитория подходит для занятия в расписании

Убираем столовую, туалет, гардероб, приемную комиссию.

Вообще здесь было бы неплохо ввести списки аудиторий для занятий и аудиторий доп. назначения(учебный офис, деканат, столовая и прочее)

ILIKE - игнорирует регистр

```

1. CREATE OR REPLACE FUNCTION session.check_audience_suitable()
2. RETURNS TRIGGER AS $$
3. DECLARE
4.     audience_name VARCHAR;
5. BEGIN
6.     SELECT audienc_number INTO audience_name
7.     FROM session.audience
8.     WHERE id_audience = NEW.fk_id_audience;
9.     IF audience_name ILIKE '%столовая%' OR
10.        audience_name ILIKE '%гардероб%' OR
11.        audience_name ILIKE '%туалет%' OR
12.        audience_name ILIKE '%приемная%' THEN
13.         RAISE EXCEPTION 'Аудитория "%" - неучебное помещение', audience_name;
14.     END IF;

```

```

15.
16.     RETURN NEW;
17. END;
18. $$ LANGUAGE plpgsql;
19.
20. CREATE TRIGGER trg_check_audience_suitable
21.     BEFORE INSERT OR UPDATE ON session.schedule
22.     FOR EACH ROW
23.     EXECUTE FUNCTION session.check_audience_suitable();

```

Попробуем внести в расписание для дисциплины неподходящую аудиторию.

Неподходящие аудитории

```

postgres=# SELECT
postgres=#     id_audience,
postgres=#     audienc_number AS название_аудитории,
postgres=#     capacity AS вместимость,
postgres=#     audience_floor AS этаж,
postgres=#     area_name AS корпус
postgres=# FROM session.audience a
postgres=# LEFT JOIN session.area ar ON a.fk_area = ar.id_area
postgres=# WHERE audienc_number ILIKE '%столовая%'
postgres=#     OR audienc_number ILIKE '%гардероб%'
postgres=#     OR audienc_number ILIKE '%туалет%'
postgres=#     OR audienc_number ILIKE '%приемная%'
postgres=# ORDER BY id_audience;
 id_audience | название_аудитории | вместимость | этаж | корпус
-----+-----+-----+-----+-----
          25 | Столовая           |          300 |     1 | Главный корпус
          26 | Туалет              |           10 |     1 | Главный корпус
          27 | Приемная комиссия |           20 |     1 | Главный корпус
          28 | Гардероб            |           40 |     1 | Главный корпус
          35 | Гардероб            |           80 |     1 |
          36 | Туалет (ж)         |           10 |     1 |
(6 строк)

```

```

postgres=# INSERT INTO session.schedule (
postgres=#     time_start, time_end, date,
postgres=#     fk_id_discipline_in_the_syllabus, fk_id_audience,
postgres=#     fk_id_teacher, fk_group_number
postgres=# ) VALUES (
postgres=#     '10:00:00+03', '11:30:00+03', CURRENT_DATE + INTERVAL '10 day
s',
postgres=#     5, 25, '550e8400-e29b-41d4-a716-446655440000',
postgres=#     'c6366216-e6a4-45ba-aeaf-9aa90e4e73ca'
postgres=# );
ОШИБКА: Аудитория "Столовая" – неучебное помещение
КОНТЕКСТ: функция PL/pgSQL session.check_audience_suitable(), строка 12, оп
ератор RAISE
postgres=#

```

А с учебной аудиторией все работает корректно

```

postgres=# INSERT INTO session.schedule (
postgres(#   time_start, time_end, date,
postgres(#   fk_id_discipline_in_the_syllabus, fk_id_audience,
postgres(#   fk_id_teacher, fk_group_number
postgres(# ) VALUES (
postgres(#   '15:30:00+03', '17:00:00+03', CURRENT_DATE + INTERVAL '10 day
s',
postgres(#   10, 30, '550e8400-e29b-41d4-a716-446655440000',
postgres(#   'c6366216-e6a4-45ba-aeaf-9aa90e4e73ca'
postgres(# );
INSERT 0 1

```

11) Проверка того, что преподаватель назначен не на одно и то же время

Не могла не ввести этот триггер, по моему мнению, он важен, так как назначение одного и того же преподавателя у нескольких групп в одно и то же время, довольно частое явление в учебных заведениях.

```

1. CREATE OR REPLACE FUNCTION session.check_teacher_available()
2. RETURNS TRIGGER AS $$
3. BEGIN
4.     IF EXISTS (
5.         SELECT 1 FROM session.schedule
6.         WHERE fk_id_teacher = NEW.fk_id_teacher
7.         AND date = NEW.date
8.         AND time_start = NEW.time_start
9.         AND (TG_OP = 'INSERT' OR id_schedule != NEW.id_schedule)
10.    ) THEN
11.        RAISE EXCEPTION 'Преподаватель уже занят в это время (дата: %, время: %)',
12.        NEW.date, NEW.time_start;
13.    END IF;
14.
15.    RETURN NEW;
16. END;
17. $$ LANGUAGE plpgsql;
18.
19. CREATE TRIGGER trg_check_teacher_available
20. BEFORE INSERT OR UPDATE ON session.schedule
21. FOR EACH ROW
22. EXECUTE FUNCTION session.check_teacher_available();

```

```

postgres=# select * from session.teacher;

```

job_title	teacher_surname	teacher_name	teacher_patronymic	agreement	id_teacher	isu
Профессор	Иванов	Сергей	Александрович	123456	550e8400-e29b-41d4-a716-446655440000	445566
Доцент	Петрова	Ольга	Викторовна	234567	550e8400-e29b-41d4-a716-446655440001	556677
Старший преподаватель	Сидоров	Андрей	Петрович	345678	550e8400-e29b-41d4-a716-446655440002	667788
Преподаватель	Смирнова	Елена	Дмитриевна	456789	550e8400-e29b-41d4-a716-446655440003	778899
Профессор	Кузнецов	Дмитрий	Иванович	567890	550e8400-e29b-41d4-a716-446655440004	889900
Заведующий кафедрой	Михайлова	Анна	Сергеевна	678901	550e8400-e29b-41d4-a716-446655440005	990011

```

postgres=# INSERT INTO session.schedule (
postgres(#      time_start, time_end, date,
postgres(#      fk_id_discipline_in_the_syllabus, fk_id_audience,
postgres(#      fk_id_teacher, fk_group_number
postgres(# ) VALUES (
postgres(#      '10:00:00+03', '11:30:00+03', CURRENT_DATE + INTERVAL '20 days',
postgres(#      11, 40, '550e8400-e29b-41d4-a716-446655440002',
postgres(#      'c6366216-e6a4-45ba-aeaf-9aa90e4e73ca'
postgres(# ) RETURNING id_schedule, fk_id_teacher, time_start, date;
 id_schedule |          fk_id_teacher          | time_start |      date
-----+-----+-----+-----
        65 | 550e8400-e29b-41d4-a716-446655440002 | 10:00:00+03 | 2026-05-21
(1 строка)

```

А теперь в другой аудитории в это же время назначим этого же преподавателя.

```

postgres=# INSERT INTO session.schedule (
postgres(#      time_start, time_end, date,
postgres(#      fk_id_discipline_in_the_syllabus, fk_id_audience,
postgres(#      fk_id_teacher, fk_group_number
postgres(# ) VALUES (
postgres(#      '10:00:00+03', '11:30:00+03', CURRENT_DATE + INTERVAL '20 day
s',
postgres(#      11, 32, '550e8400-e29b-41d4-a716-446655440002',
postgres(#      'c6366216-e6a4-45ba-aeaf-9aa90e4e73ca'
postgres(# ) RETURNING id_schedule, fk_id_teacher, time_start, date;
ОШИБКА: Преподаватель уже занят в это время (дата: 2026-05-21, время: 10:00
:00+03)
КОНТЕКСТ: функция PL/pgSQL session.check_teacher_available(), строка 10, оп
ератор RAISE

```

Логирование

- запись того, кто и когда провел изменения в таблице

Как работает:

- 1) выполняем действие INSERT, UPDATE, DELETE в определенной таблице(например, teacher)
- 2) Триггер автоматически срабатывает
- 3) Триггер вызывает специальную функцию
- 4) Функция записывает информацию о действии в отдельную таблицу логов
- 5) Основная операция продолжается как обычно

Для таблицы **teacher**:

Создадим таблицу для логов как в методичке

```

1. CREATE TABLE IF NOT EXISTS session.teacher_log (
2.     text TEXT,
3.     added TIMESTAMP WITHOUT TIME ZONE DEFAULT NOW()
4. );

```

```

postgres=# CREATE TABLE IF NOT EXISTS session.teacher_log (
postgres(#      text TEXT,
postgres(#      added TIMESTAMP WITHOUT TIME ZONE DEFAULT NOW())
postgres(# );
CREATE TABLE
postgres=#

```

Создаем функцию триггера add_teacher_to_log()

Только в данном случае, я решила убрать действие REMOVE, так как я предполагаю, что в учебном заведении хранится история работников и студентов.

Предварительный просмотр кода на веб-странице

```

1. CREATE OR REPLACE FUNCTION session.add_teacher_to_log()
2. RETURNS TRIGGER AS $$
3. DECLARE
4.     mstr VARCHAR(30);
5.     astr VARCHAR(100);
6.     retstr VARCHAR(254);
7. BEGIN
8.     IF TG_OP = 'INSERT' THEN
9.         astr := NEW.id_teacher;
10.        mstr := 'Add new teacher ';
11.        retstr := mstr || astr;
12.        INSERT INTO session.teacher_log (text, added) VALUES (retstr, NOW());
13.        RETURN NEW;
14.    ELSIF TG_OP = 'UPDATE' THEN
15.        astr := NEW.id_teacher;
16.        mstr := 'Update teacher ';
17.        retstr := mstr || astr;
18.        INSERT INTO session.teacher_log (text, added) VALUES (retstr, NOW());
19.        RETURN NEW;
20.    END IF;
21. END;
22. $$ LANGUAGE plpgsql;

```

```

postgres=# CREATE OR REPLACE FUNCTION session.add_teacher_to_log()
postgres=# RETURNS TRIGGER AS $$
postgres## DECLARE
postgres##     mstr VARCHAR(30);
postgres##     astr VARCHAR(100);
postgres##     retstr VARCHAR(254);
postgres## BEGIN
postgres##     IF TG_OP = 'INSERT' THEN
postgres##         astr := NEW.id_teacher;
postgres##         mstr := 'Add new teacher ';
postgres##         retstr := mstr || astr;
postgres##         INSERT INTO session.teacher_log (text, added) VALUES (retstr, NOW());
postgres##         RETURN NEW;
postgres##     ELSIF TG_OP = 'UPDATE' THEN
postgres##         astr := NEW.id_teacher;
postgres##         mstr := 'Update teacher ';
postgres##         retstr := mstr || astr;
postgres##         INSERT INTO session.teacher_log (text, added) VALUES (retstr, NOW());
postgres##         RETURN NEW;
postgres##     END IF;
postgres## END;
postgres## $$ LANGUAGE plpgsql;
CREATE FUNCTION

```


Создаем сам триггер

```
1. CREATE TRIGGER t_teacher
2. AFTER INSERT OR UPDATE ON session.teacher
3. FOR EACH ROW
4. EXECUTE FUNCTION session.add_teacher_to_log();
```

Здесь используем **AFTER** потому что хотим залогировать **уже совершенное действие**. Логирование не должно мешать основной операции.

Из методички:

Если вы хотите заблокировать событие типа INSERT, вы захотите выполнять действие до (BEFORE). Если вы хотите быть уверенным, что событие действительно произойдет, идеальный вариант - после (AFTER).

Давайте проверим, добавим нового преподавателя.

```
1. INSERT INTO session.teacher (
2.     id_teacher, job_title, teacher_surname, teacher_name,
3.     teacher_patronymic, agreement, isu
4. ) VALUES (
5.     gen_random_uuid(), 'Доцент', 'Васильева', 'Алиса',
6.     'Николаевна', '555555', '123456'
7. ) RETURNING id_teacher;
```

```
CREATE FUNCTION
postgres=# CREATE TRIGGER t_teacher
postgres=# AFTER INSERT OR UPDATE ON session.teacher
postgres=# FOR EACH ROW
postgres=# EXECUTE FUNCTION session.add_teacher_to_log();
CREATE TRIGGER
postgres=# INSERT INTO session.teacher (
postgres=#     id_teacher, job_title, teacher_surname, teacher_name,
postgres=#     teacher_patronymic, agreement, isu
postgres=# ) VALUES (
postgres=#     gen_random_uuid(), 'Доцент', 'Васильева', 'Алиса',
postgres=#     'Николаевна', '555555', '123456'
postgres=# ) RETURNING id_teacher;
          id_teacher
-----
ce3fb2b0-6a1e-4f03-bed4-d9f1187581c2
(1 ёЁёюр)

INSERT 0 1
```

Проверим, что запись появилась в таблице

```
postgres=# select * from session.teacher where teacher_surname = 'Васильева'
```

job_title	teacher_surname	teacher_name	teacher_patronymic	agreement	id_teacher	isu
Доцент	Васильева	Алиса	Николаевна	555555	b1ce3e2e-a418-4fa0-ad80-c49247ee5231	123456

(1 строка)

А теперь проверим таблицу логов

```
postgres=# select * from session.add_teacher_to_log();
ОШИБКА: функция "add_teacher_to_log", используемая во FROM, возвращает неподдерживаемый тип trigger
СТРОКА 1: select * from session.add_teacher_to_log();
^
postgres=# SELECT * FROM session.teacher_log ORDER BY added DESC LIMIT 5;
```

text	added
Add new teacher b1ce3e2e-a418-4fa0-ad80-c49247ee5231	2026-05-02 12:52:13.252018
Add new teacher ce3fb2b0-6a1e-4f03-bed4-d9f1187581c2	2026-05-02 12:47:40.480934

(2 строки)

Можем увидеть, что записи добавления студентов присутствует в таблице.

Можем еще сделать проверку на UPDATE

```
postgres=# Update session.teacher
postgres=# set teacher_surname = 'Лимонова'
postgres=# where id_teacher = 'b1ce3e2e-a418-4fa0-ad80-c49247ee5231';
UPDATE 1
```

```
postgres=# SELECT * FROM session.teacher_log;
```

text	added
Add new teacher ce3fb2b0-6a1e-4f03-bed4-d9f1187581c2	2026-05-02 12:47:40.480934
Add new teacher b1ce3e2e-a418-4fa0-ad80-c49247ee5231	2026-05-02 12:52:13.252018
Update teacher b1ce3e2e-a418-4fa0-ad80-c49247ee5231	2026-05-02 13:11:04.006568

(3 строки)

Для таблицы **student**:

Аналогично,

```
1. CREATE TABLE IF NOT EXISTS session.student_log (
2.     text TEXT,
3.     added TIMESTAMP WITHOUT TIME ZONE DEFAULT NOW()
4. );
```

```
1. CREATE OR REPLACE FUNCTION session.add_student_to_log()
2. RETURNS TRIGGER AS $$
3. DECLARE
```

```

4.     mstr VARCHAR(30);
5.     astr VARCHAR(100);
6.     retstr VARCHAR(254);
7. BEGIN
8.     IF TG_OP = 'INSERT' THEN
9.         astr := NEW.student_id;
10.        mstr := 'Add new student ';
11.        retstr := mstr || astr;
12.        INSERT INTO session.student_log (text, added) VALUES (retstr, NOW());
13.        RETURN NEW;
14.    ELSIF TG_OP = 'UPDATE' THEN
15.        astr := NEW.student_id;
16.        mstr := 'Update student ';
17.        retstr := mstr || astr;
18.        INSERT INTO session.student_log (text, added) VALUES (retstr, NOW());
19.        RETURN NEW;
20.    END IF;
21. END;
22. $$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER t_student
1. AFTER INSERT OR UPDATE ON session.student
2. FOR EACH ROW
3. EXECUTE FUNCTION session.add_student_to_log();

```

```

postgres=# SELECT * FROM session.student_log;

```

text	added
Add new student aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaaa	2026-05-02 13:22:20.222305

(1 строка)

Задание 2.2 (модификация)

Модифицировать триггер (триггерную функцию) на проверку корректности входа и выхода сотрудника (см. Практическое задание 1 Лабораторного практикума (Приложение)) с максимальным учетом «узких» мест некорректных данных по входу и выходу).

Перед выполнением этого задания, я решила повторно выполнить все действия практической работы.

Создаем базу данных

```
postgres=# create database lab5_modification;  
CREATE DATABASE
```

Для корректного отображения кириллицы

```
postgres=# \! chcp 1251  
Текущая кодовая страница: 1251  
postgres=# |
```

Посмотрим список всех активных бд

```
postgres=# \list
```

Список ба

з данных

Имя	Владелец	Кодировка	Провайдер локали	LC_COLLAT
LC_CTYPE	Локаль	Правила ICU	Права доступа	
db	postgres	UTF8	libc	Russian_Russi
a.1251 Russian_Russia.1251	postgres	UTF8	libc	Russian_Russi
emp_time	postgres	UTF8	libc	Russian_Russi
a.1251 Russian_Russia.1251	postgres	UTF8	libc	Russian_Russi
lab5_modification	postgres	UTF8	libc	Russian_Russi
a.1251 Russian_Russia.1251	postgres	UTF8	libc	Russian_Russi
postgres	postgres	UTF8	libc	Russian_Russi
a.1251 Russian_Russia.1251	postgres	UTF8	libc	Russian_Russi
proglab	postgres	UTF8	libc	Russian_Russi
a.1251 Russian_Russia.1251	postgres	UTF8	libc	Russian_Russi
template0	postgres	UTF8	libc	Russian_Russi
a.1251 Russian_Russia.1251			=c/postgres	+
			postgres=Ct/postgres	
template1	postgres	UTF8	libc	Russian_Russi
a.1251 Russian_Russia.1251			=c/postgres	+
			postgres=Ct/postgres	
test	postgres	UTF8	libc	Russian_Russi
a.1251 Russian_Russia.1251				

(8 строк)

Подключились к нужной нам БД

```
postgres=# \c lab5_modification
Вы подключены к базе данных "lab5_modification" как пользователь "postgres".
```

Пример 1. Таймер для учета работы сотрудников

Задание 1:

1. Создайте БД emp_time. Используйте консоль psql.
2. Проверьте список активных БД.
3. Подключитесь к БД emp_time.
4. Создайте таблицы БД со следующей структурой:

Таблица employee

id (тип serial, первичный ключ)
username (тип varchar)

Таблица time_punch

id (тип serial, первичный ключ)
employee_id (тип int, обязательное, внешний ключ: соответствует ключу employee.id)
is_out_punch (тип boolean, обязательное, значение по умолчанию false)
punch_time (тип timestamp, обязательное, значение по умолчанию now()).

```
lab5_modification=# create table employee(
lab5_modification(# id serial primary key,
lab5_modification(# username varchar(100) not null);
CREATE TABLE
```

```
lab5_modification=# CREATE TABLE time_punch (
lab5_modification(#      id SERIAL PRIMARY KEY,
lab5_modification(#      employee_id INTEGER NOT NULL REFERENCES employee(id)
lab5_modification(#      is_out_punch BOOLEAN NOT NULL DEFAULT false,
lab5_modification(#      punch_time TIMESTAMP NOT NULL DEFAULT now()
lab5_modification(# );
CREATE TABLE
```

Вставьте данные в таблицы:

1. Вставьте в таблицу сотрудников строку для сотрудника с именем Михаил.
2. Вставьте в таблицу учета времени две строки: вход Михаил в 10.00 2021-01-01 и выход в 11.30 2021-01-01.
3. Проверьте содержимое таблиц.

Вычисление рабочего времени

Запрос позволяет на каждый выход найти соответствующий вход:

```
select tp1.punch_time - tp2.punch_time as time_worked
from time_punch tp1 join time_punch tp2
on tp2.id = (select tps.id from time_punch tps
where tps.id < tp1.id and
tps.employee_id = tp1.employee_id and
not tps.is_out_punch
order by tps.id desc limit 1
)
where tp1.employee_id = 1 and tp1.is_out_punch;
```

10

```
lab5_modification=# insert into employee (username) values ('Михаил');
INSERT 0 1
```

```
lab5_modification=# insert into time_punch (employee_id, is_out_punch, punch
_time)
lab5_modification-# values ((select id from employee where username = 'Михаи
л'), false, '2021-01-01 10:00:00');
INSERT 0 1
lab5_modification=# INSERT INTO time_punch (employee_id, is_out_punch, punch
_time)
lab5_modification-# VALUES ((SELECT id FROM employee WHERE username = 'Михаи
л'), true, '2021-01-01 11:30:00');
INSERT 0 1
```

```
lab5_modification=# select * from employee;
```

id	username
1	Михаил

(1 строка)

```
lab5_modification=# select * from time_punch;
```

id	employee_id	is_out_punch	punch_time
1	1	f	2021-01-01 10:00:00
2	1	t	2021-01-01 11:30:00

(2 строки)

Вычислим рабочее время Михаила

```
lab5_modification=# select tp1.punch_time - tp2.punch_time as time_worked
lab5_modification-# from time_punch tp1 join time_punch tp2
lab5_modification-# on tp2.id = (select tps.id from time_punch tps
lab5_modification(# where tps.id < tp1.id and
lab5_modification(# tps.employee_id = tp1.
lab5_modification( employee_id and
lab5_modification(# not tps.is_out_punc
lab5_modification( h
lab5_modification(# order by tps.id desc limit 1
lab5_modification(# )
lab5_modification-# where tp1.employee_id = 1 and tp1.is_out_punch;
lab5_modification-# time_worked
```

01:30:00

(1 строка)

Задание 2:

Проверьте работу запроса.

Примечание.

Одна из проблем в этой схеме состоит в том, что возможно вставить несколько "входов" или "выходов" подряд. С созданным запросом это приведет к неоднозначности, которая может привести к неточным расчетам и зарплате сотрудников – больше или меньше, чем они должны были бы получить.

Триггер INSERT BEFORE: сохранение целостности данных

Требуется то, что не позволит нарушить шаблон вход/выход. К сожалению, ограничения check только отслеживают вставляемую или обновляемую строку и не могут учитывать данные из других строк.

Создадим триггер для предотвращения события INSERT, которое нарушает шаблон. Сначала создадим "триггерную функцию". Эта функция есть то, что будет выполнять триггер при наступлении события.

Триггерная функция создается как обычная функция PostgreSQL за тем исключением, что возвращает *триггер*.

```
create or replace function fn_check_time_punch() returns
trigger as $psql$
begin
    if new.is_out_punch = (
        select tps.is_out_punch
        from time_punch tps
        where tps.employee_id = new.employee_id
        order by tps.id desc limit 1
    ) then
        return null;
    end if;
    return new;
end;
$psql$ language plpgsql;
```

```
lab5_modification=# create or replace function fn_check_time_punch() returns
lab5_modification$# trigger as $psql$
lab5_modification$# begin
lab5_modification$#     if new.is_out_punch = (
lab5_modification$#         select tps.is_out_punch
lab5_modification$#         from time_punch tps
lab5_modification$#         where tps.employee_id = new.employee_id
lab5_modification$#         order by tps.id desc limit 1
lab5_modification$#     ) then
lab5_modification$#         return null;
lab5_modification$#     end if;
lab5_modification$#     return new;
lab5_modification$# end;
lab5_modification$# $psql$ language plpgsql;
lab5_modification$# CREATE FUNCTION
```

```
lab5_modification=# create trigger check_time_punch before insert on time_pu
lab5_modification$# nch
lab5_modification$# for each row execute procedure fn_check_time_punch();
lab5_modification$# CREATE TRIGGER
```


Задание 3:

1. Проверьте работу триггера на корректных и некорректных данных по входу и выходу сотрудника.
2. Найдите “узкие” места в работе триггера.

11

3. Выполните запрос на просмотр данных в таблице учета времени.

```
lab5_modification=# INSERT INTO time_punch (employee_id, is_out_punch, punch_time)
lab5_modification=# VALUES (1, false, '2024-01-15 09:00:00');
INSERT 0 1
lab5_modification=# INSERT INTO time_punch (employee_id, is_out_punch, punch_time)
lab5_modification=# VALUES (1, true, '2024-01-15 18:00:00');
INSERT 0 1
lab5_modification=# INSERT INTO time_punch (employee_id, is_out_punch, punch_time)
lab5_modification=# VALUES (1, false, '2024-01-16 09:00:00');
INSERT 0 1
lab5_modification=# select * from time_punch;
 id | employee_id | is_out_punch |      punch_time
-----+-----+-----+-----
  5 |           1 | f           | 2024-01-15 09:00:00
  6 |           1 | t           | 2024-01-15 18:00:00
  7 |           1 | f           | 2024-01-16 09:00:00
(3 строки)

lab5_modification=# INSERT INTO time_punch (employee_id, is_out_punch, punch_time)
lab5_modification=# VALUES (1, false, '2024-01-16 10:00:00');
INSERT 0 0
lab5_modification=# select * from time_punch;
 id | employee_id | is_out_punch |      punch_time
-----+-----+-----+-----
  5 |           1 | f           | 2024-01-15 09:00:00
  6 |           1 | t           | 2024-01-15 18:00:00
  7 |           1 | f           | 2024-01-16 09:00:00
(3 строки)
```

Теперь поищем уязвимые места в коде

- 1) Первая запись может быть выходом

```
lab5_modification=# DELETE FROM time_punch;
DELETE 5
lab5_modification=# select * from time_punch;
 id | employee_id | is_out_punch | punch_time
-----+-----+-----+-----
(0 строк)

lab5_modification=# INSERT INTO time_punch (employee_id, is_out_punch, punch_time)
lab5_modification=# VALUES (1, true, '2024-01-16 10:00:00');
INSERT 0 1
```

```
1. create or replace function fn_check_time_punch() returns trigger as $psql$
2. declare
3.     last_out boolean;
4. begin
5.     select is_out_punch into last_out
6.     from time_punch tps
7.     where tps.employee_id = new.employee_id
8.     order by tps.id desc limit 1;
9.     if not found then
10.         if new.is_out_punch = true then
11.             raise exception 'Нельзя выйти изначально!';
12.         end if;
13.         return new;
14.     end if;
15.
16.     if new.is_out_punch = last_out then
17.         return null;
18.     end if;
19.
20.     return new;
21. end;
22. $psql$ language plpgsql;
```

```
lab5_modification=# delete from time_punch;
DELETE 1
lab5_modification=# select * from time_punch;
 id | employee_id | is_out_punch | punch_time
-----+-----+-----+-----
(0 строк)

lab5_modification=# INSERT INTO time_punch (employee_id, is_out_punch, punch_time)
lab5_modification=# VALUES (1, true, '2024-01-16 10:00:00');
ОШИБКА: Нельзя выйти изначально!
КОНТЕКСТ: функция PL/pgSQL fn_check_time_punch(), строка 11, оператор RAISE
lab5_modification=# |
```

Триггер выполняется при первой записи сотрудника “выход”.

3) Отрицательное время: выход раньше входа

```

lab5_modification=# INSERT INTO time_punch (employee_id, is_out_punch, punch_time)
lab5_modification=# VALUES (1, false, '2024-01-16 10:00:00');
INSERT 0 1
lab5_modification=# INSERT INTO time_punch (employee_id, is_out_punch, punch_time)
lab5_modification=# VALUES (1, true, '2024-01-16 09:00:00');
INSERT 0 1
lab5_modification=# select * from time_punch;
 id | employee_id | is_out_punch | punch_time
-----+-----+-----+-----
 14 |           1 | f           | 2024-01-16 10:00:00
 15 |           1 | t           | 2024-01-16 09:00:00
(2 строки)

```

Поменяю значение last_out на last_record типа record(будем смотреть по записи)

```

1. create or replace function fn_check_time_punch() returns trigger as $psql$
2. declare
3.     last_record record;
4. begin
5.     select id, is_out_punch, punch_time into last_record
6.     from time_punch tps
7.     where tps.employee_id = new.employee_id
8.     order by tps.id desc limit 1;
9.
10.    if not found then
11.        if new.is_out_punch = true then
12.            raise exception 'Нельзя выйти изначально!';
13.        end if;
14.        return new;
15.    end if;
16.    if new.is_out_punch = last_record.is_out_punch then
17.        return null;
18.    end if;
19.
20.    -- время неотрицательное
21.    if new.is_out_punch = true then
22.        if new.punch_time <= last_record.punch_time then
23.            raise exception 'Время выхода не должно быть позе времени входа!';
24.        end if;
25.    end if;
26.
27.    return new;
28. end;
29. $psql$ language plpgsql;

```

```

lab5_modification=# INSERT INTO time_punch (employee_id, is_out_punch, punch_time)
lab5_modification=# VALUES (1, false, '2024-01-16 10:00:00');
INSERT 0 1
lab5_modification=# INSERT INTO time_punch (employee_id, is_out_punch, punch_time)
lab5_modification=# VALUES (1, true, '2024-01-16 08:00:00');
ОШИБКА: Время выхода не должно быть позе времени входа!
КОНТЕКСТ: функция PL/pgSQL fn_check_time_punch(), строка 23, оператор RAISE
lab5_modification=#

```

3) Могу отмечаться в будущем времени

```
lab5_modification=# VALUES (1, false, '2027-01-16 10:00:00');
INSERT 0 1
lab5_modification=# select * from time_punch;
 id | employee_id | is_out_punch | punch_time
-----+-----+-----+-----
  5 |           1 | f            | 2024-01-15 09:00:00
  6 |           1 | t            | 2024-01-15 18:00:00
  7 |           1 | f            | 2024-01-16 09:00:00
 10 |           1 | t            | 2026-05-02 14:31:06.791065
 11 |           1 | f            | 2027-01-16 10:00:00
(5 строк)
```

```
1. create or replace function fn_check_time_punch() returns trigger as $psql$
2. declare
3.     last_record record;
4. begin
5.     if new.punch_time > now() then
6.         raise exception 'Пользователь не может отмечаться в будущем!';
7.     end if;
8.
9.     select id, is_out_punch, punch_time into last_record
10.    from time_punch tps
11.   where tps.employee_id = new.employee_id
12.  order by tps.id desc limit 1;
13.
14.     if not found then
15.         if new.is_out_punch = true then
16.             raise exception 'Нельзя выйти изначально!';
17.         end if;
18.         return new;
19.     end if;
20.     if new.is_out_punch = last_record.is_out_punch then
21.         return null;
22.     end if;
23.
24.     -- время неотрицательное
25.     if new.is_out_punch = true then
26.         if new.punch_time <= last_record.punch_time then
27.             raise exception 'Время выхода не должно быть позе времени входа!';
28.         end if;
29.     end if;
30.
31.     return new;
32. end;
33. $psql$ language plpgsql;
```

```
lab5_modification=# INSERT INTO time_punch (employee_id, is_out_punch, punch_time)
lab5_modification=# VALUES (1, false, '2028-01-16 10:00:00');
ОШИБКА: Пользователь не может отмечаться в будущем!
КОНТЕКСТ: функция PL/pgSQL fn_check_time_punch(), строка 6, оператор RAISE
lab5_modification=#
```

4) Смена не должна превышать 12 часов

```
DELETE 1
lab5_modification=# INSERT INTO time_punch (employee_id, is_out_punch, p
unch_time)
lab5_modification=# VALUES (1, false, '2024-01-16 10:00:00');
INSERT 0 1
lab5_modification=# INSERT INTO time_punch (employee_id, is_out_punch, p
unch_time)
lab5_modification=# VALUES (1, true, '2024-01-19 10:00:00');
INSERT 0 1
lab5_modification=# select * from time_punch;
 id | employee_id | is_out_punch |      punch_time
-----+-----+-----+-----
 23 |           1 | f           | 2024-01-16 10:00:00
 24 |           1 | t           | 2024-01-19 10:00:00
(2 строки)
```

interval - тип данных, который хранит промежуток времени

```
1. create or replace function fn_check_time_punch() returns trigger as $psql$
2. declare
3.     last_record record;
4.     time_diff interval;
5. begin
6.     if new.punch_time > now() then
7.         raise exception 'Пользователь не может отмечаться в будущем!';
8.     end if;
9.
10.    select id, is_out_punch, punch_time into last_record
11.    from time_punch tps
12.    where tps.employee_id = new.employee_id
13.    order by tps.id desc limit 1;
14.
15.    if not found then
16.        if new.is_out_punch = true then
17.            raise exception 'Нельзя выйти изначально!';
18.        end if;
19.        return new;
20.    end if;
21.    if new.is_out_punch = last_record.is_out_punch then
22.        return null;
23.    end if;
24.
25.    -- время неотрицательное
26.    if new.is_out_punch = true then
27.        if new.punch_time <= last_record.punch_time then
28.            raise exception 'Время выхода не должно быть позже времени входа!';
29.        end if;
30.
31.
32.    time_diff = new.punch_time - last_record.punch_time;
33.    if time_diff > interval '24 hours' then
34.        raise exception 'Рабочий день не может длиться больше 12 часов!';
35.    end if;
36.    end if;
```

```

37.
38.     return new;
39. end;
40. $psql$ language plpgsql;

```

```

lab5_modification=# INSERT INTO time_punch (employee_id, is_out_punch, punch
_time)
lab5_modification=# VALUES (1, false, '2024-01-16 10:00:00');
INSERT 0 1
lab5_modification=# INSERT INTO time_punch (employee_id, is_out_punch, punch
_time)
lab5_modification=# VALUES (1, true, '2024-01-19 10:00:00');
ОШИБКА: Рабочий день не может длиться больше 12 часов!
КОНТЕКСТ: функция PL/pgSQL fn_check_time_punch(), строка 34, оператор RAISE
lab5_modification=# |

```

Я еще добавила raise exception в код из методички в случай, где проверяется 2 входа или 2 выхода

```

1. create or replace function fn_check_time_punch() returns trigger as $psql$
2. declare
3.     last_record record;
4.     time_diff interval;
5. begin
6.     if new.punch_time > now() then
7.         raise exception 'Пользователь не может отмечаться в будущем!';
8.     end if;
9.
10.    select id, is_out_punch, punch_time into last_record
11.    from time_punch tps
12.    where tps.employee_id = new.employee_id
13.    order by tps.id desc limit 1;
14.
15.    if not found then
16.        if new.is_out_punch = true then
17.            raise exception 'Нельзя выйти изначально!';
18.        end if;
19.        return new;
20.    end if;
21.    if new.is_out_punch = last_record.is_out_punch then
22.        raise exception 'Невозможно записать 2 входа или 2 выхода одного сотрудника!';
23.    end if;
24.
25.    -- время неотрицательное
26.    if new.is_out_punch = true then
27.        if new.punch_time <= last_record.punch_time then
28.            raise exception 'Время выхода не должно быть позже времени входа!';
29.        end if;
30.
31.
32.    time_diff = new.punch_time - last_record.punch_time;
33.    if time_diff > interval '24 hours' then
34.        raise exception 'Рабочий день не может длиться больше 12 часов!';
35.    end if;
36. end if;
37.
38.     return new;
39. end;
40. $psql$ language plpgsql;

```

5) Отметка о входе или выходе не должна быть зарегистрирована позже начала существования предприятия

Пусть предприятие начало действовать в 1995 году.

```
1. create or replace function fn_check_time_punch() returns trigger as $psql$
2. declare
3.     last_record record;
4.     time_diff interval;
5. begin
6.     if new.punch_time > now() then
7.         raise exception 'Пользователь не может отмечаться в будущем!';
8.     end if;
9.     if new.punch_time < '1995-01-01' then
10.        raise exception 'Неверная дата. Указанная дата раньше 01.01.1995!';
11.    end if;
12.
13.    select id, is_out_punch, punch_time into last_record
14.    from time_punch tps
15.    where tps.employee_id = new.employee_id
16.    order by tps.id desc limit 1;
17.
18.    if not found then
19.        if new.is_out_punch = true then
20.            raise exception 'Нельзя выйти изначально!';
21.        end if;
22.        return new;
23.    end if;
24.    if new.is_out_punch = last_record.is_out_punch then
25.        raise exception 'Невозможно записать 2 входа или 2 выхода одного сотрудника!';
26.    end if;
27.
28.    -- время неотрицательное
29.    if new.is_out_punch = true then
30.        if new.punch_time <= last_record.punch_time then
31.            raise exception 'Время выхода не должно быть позже времени входа!';
32.        end if;
33.
34.
35.        time_diff = new.punch_time - last_record.punch_time;
36.        if time_diff > interval '24 hours' then
37.            raise exception 'Рабочий день не может длиться больше 12 часов!';
38.        end if;
39.    end if;
40.
41.    return new;
42. end;
43. $psql$ language plpgsql;
```

Корректную дату записывает без проблем

```
lab5_modification=# INSERT INTO time_punch (employee_id, is_out_punch, punch_time)
lab5_modification=# VALUES (1, false, '2025-01-16 10:00:00');
INSERT 0 1
```

А вот здесь срабатывает триггер

```
DELETE FROM time_punch WHERE employee_id=1;
lab5_modification=# INSERT INTO time_punch (employee_id, is_out_punch, punch
_time)
lab5_modification=# VALUES (1, false, '988-01-16 10:00:00');
ОШИБКА: Неверная дата. Указанная дата раньше 01.01.1995!
КОНТЕКСТ: функция PL/pgSQL fn_check_time_punch(), строка 10, оператор RAISE
lab5_modification=#
```


ВЫВОД

В лабораторной работе я овладела практическими навыками создания и использования процедур, функций и триггеров в базе данных PostgreSQL. При выполнении понадобилось ПО: СУБД PostgreSQL, SQL Shell (psql).

Были выполнены поставленные практические задачи:

- 1) Созданы 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР 2 вариант 2).
- 2) Создала авторские процедуры
- 3) Модифицировала код триггера (триггерной функции) на проверку корректности входа и выхода сотрудника (см. Практическое задание 1 Лабораторного практикума 3) с максимальным учетом “узких” мест некорректных данных по входу и выходу.

К каждому заданию были приложены фрагменты кода, а также подтверждающие скриншоты.